HW7 CART, RandomForests and Logistic Regression

Question 10.1

Using the same crime data set uscrime.txt as in Questions 8.2 and 9.1, find the best model you can using (a) a regression tree model, and (b) a random forest model. In R, you can use the tree package or the rpart package, and the randomForest package. For each model, describe one or two qualitative takeaways you get from analyzing the results (i.e., don't just stop when you have a good model, but interpret it too).

General Observation

```
library(dplyr)
library(ggplot2)
library(gridExtra)
library(kableExtra)
library(tidyr)
library(stats)
library(caret)
library(ggcorrplot)
library(rpart)
library(rpart.plot)
library(randomForest)
library(pROC)
set.seed(73)
crime<-read.delim('uscrime.txt',header = T)
#data description
head(crime)
```

```
##      M So   Ed  Po1  Po2    LF   M.F Pop   NW    U1  U2 Wealth Ineq     Prob
## 1 15.1  1  9.1  5.8  5.6 0.510  95.0  33 30.1 0.108 4.1   3940 26.1 0.084602
## 2 14.3  0 11.3 10.3  9.5 0.583 101.2  13 10.2 0.096 3.6   5570 19.4 0.029599
## 3 14.2  1  8.9  4.5  4.4 0.533  96.9  18 21.9 0.094 3.3   3180 25.0 0.083401
## 4 13.6  0 12.1 14.9 14.1 0.577  99.4 157  8.0 0.102 3.9   6730 16.7 0.015801
## 5 14.1  0 12.1 10.9 10.1 0.591  98.5  18  3.0 0.091 2.0   5780 17.4 0.041399
## 6 12.1  0 11.0 11.8 11.5 0.547  96.4  25  4.4 0.084 2.9   6890 12.6 0.034201
##      Time Crime
## 1 26.2011   791
## 2 25.2999  1635
## 3 24.3006   578
## 4 29.9012  1969
## 5 21.2998  1234
## 6 20.9995   682
```

All the columns' descriptions are listed here:

Effect of Punishment Regimes on Crime Rates

Keywords: multiple regression, best subsets, causality.

Description

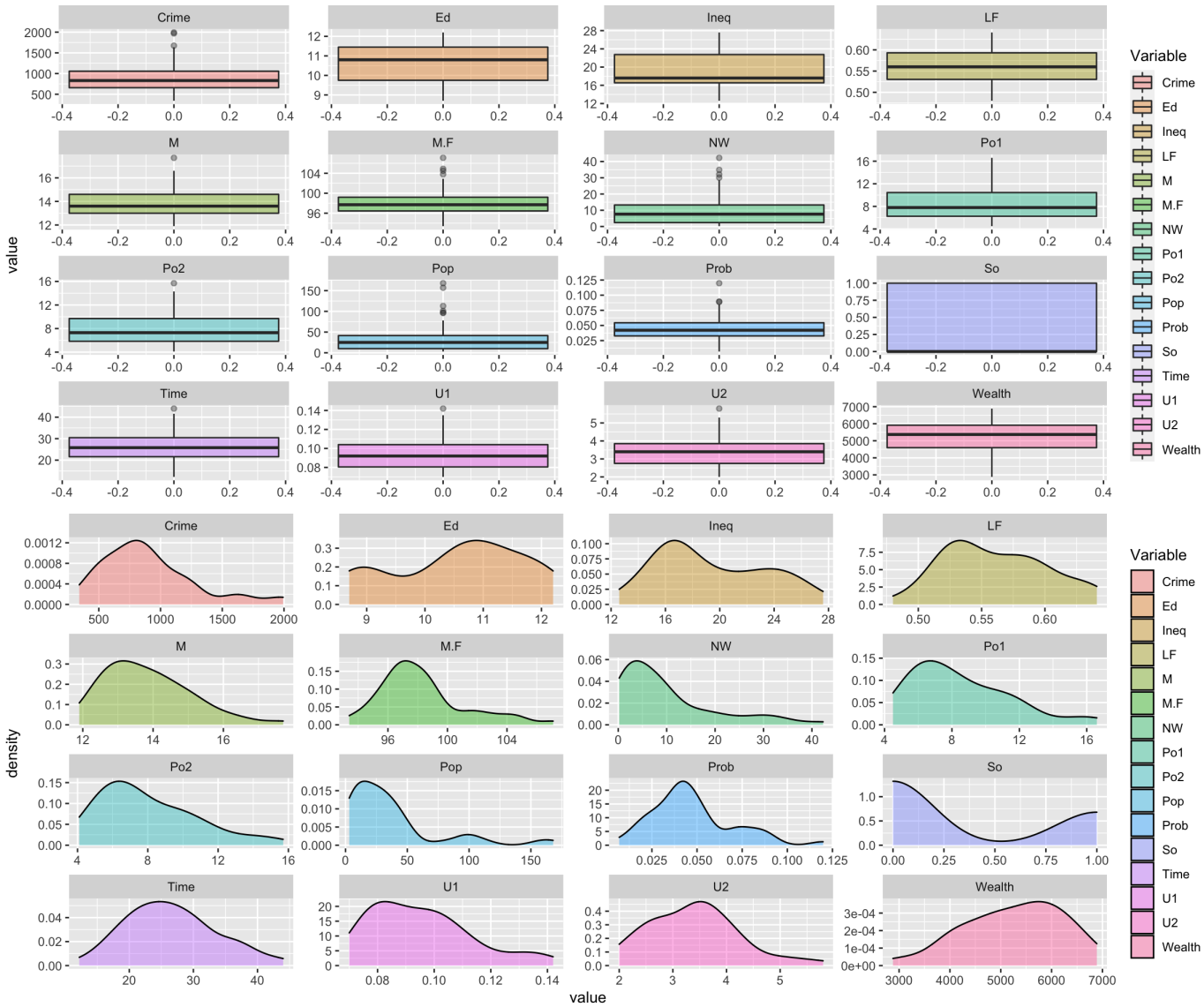
Criminologists are interested in the effect of punishment regimes on crime rates. This has been studied using aggregate data on 47 states of the USA for 1960. The data set contains the following columns:

Variable	Description
M	percentage of males aged 14–24 in total state population
So	indicator variable for a southern state
Ed	mean years of schooling of the population aged 25 years or over
Po1	per capita expenditure on police protection in 1960
Po2	per capita expenditure on police protection in 1959
LF	labour force participation rate of civilian urban males in the age-group 14-24
M.F	number of males per 100 females
Pop	state population in 1960 in hundred thousands
NW	percentage of nonwhites in the population
U1	unemployment rate of urban males 14–24
U2	unemployment rate of urban males 35–39
Wealth	wealth: median value of transferable assets or family income
Ineq	income inequality: percentage of families earning below half the median income
Prob	probability of imprisonment: ratio of number of commitments to number of offenses
Time	average time in months served by offenders in state prisons before their first release
Crime	crime rate: number of offenses per 100,000 population in 1960

```
summary(crime)
```

```
##        M               So               Ed             Po1
##  Min.   :11.90   Min.   :0.0000   Min.   : 8.70   Min.   : 4.50
##  1st Qu.:13.00   1st Qu.:0.0000   1st Qu.: 9.75   1st Qu.: 6.25
##  Median :13.60   Median :0.0000   Median :10.80   Median : 7.80
##  Mean   :13.86   Mean   :0.3404   Mean   :10.56   Mean   : 8.50
##  3rd Qu.:14.60   3rd Qu.:1.0000   3rd Qu.:11.45   3rd Qu.:10.45
##  Max.   :17.70   Max.   :1.0000   Max.   :12.20   Max.   :16.60
##       Po2               LF              M.F              Pop
##  Min.   : 4.100   Min.   :0.4800   Min.   : 93.40   Min.   :  3.00
##  1st Qu.: 5.850   1st Qu.:0.5305   1st Qu.: 96.45   1st Qu.: 10.00
##  Median : 7.300   Median :0.5600   Median : 97.70   Median : 25.00
##  Mean   : 8.023   Mean   :0.5612   Mean   : 98.30   Mean   : 36.62
##  3rd Qu.: 9.700   3rd Qu.:0.5930   3rd Qu.: 99.20   3rd Qu.: 41.50
##  Max.   :15.700   Max.   :0.6410   Max.   :107.10   Max.   :168.00
##        NW               U1                U2            Wealth
##  Min.   : 0.20   Min.   :0.07000   Min.   :2.000   Min.   :2880
##  1st Qu.: 2.40   1st Qu.:0.08050   1st Qu.:2.750   1st Qu.:4595
##  Median : 7.60   Median :0.09200   Median :3.400   Median :5370
##  Mean   :10.11   Mean   :0.09547   Mean   :3.398   Mean   :5254
##  3rd Qu.:13.25   3rd Qu.:0.10400   3rd Qu.:3.850   3rd Qu.:5915
##  Max.   :42.30   Max.   :0.14200   Max.   :5.800   Max.   :6890
##       Ineq             Prob              Time           Crime
##  Min.   :12.60   Min.   :0.00690   Min.   :12.20   Min.   : 342.0
##  1st Qu.:16.55   1st Qu.:0.03270   1st Qu.:21.60   1st Qu.: 658.5
##  Median :17.60   Median :0.04210   Median :25.80   Median : 831.0
##  Mean   :19.40   Mean   :0.04709   Mean   :26.60   Mean   : 905.1
##  3rd Qu.:22.75   3rd Qu.:0.05445   3rd Qu.:30.45   3rd Qu.:1057.5
##  Max.   :27.60   Max.   :0.11980   Max.   :44.00   Max.   :1993.0
```

From above we can see there are 47 observations (total 47 states in the US) as rows and 16 variables as columns in the uscrime.txt data set. The scales of each column are very different. Let's visualize their distribution by box whisker plot and density plot.



From the plot, we can see all the predictors' scales, distributions are very different. Most of them are not normally distributed, only Time and Wealth columns seem like normal distributions. All others are skewed. Furthermore, predictor So is a binary variable. If it is one of the predictors that we choose to fit in a model based on the numeric values of the predictors, we will need to scale the whole data set but reserve it without scaling. However, the decision tree and the random forest are not sensitive for scaling, so we are good to use the original dataset.

```
# check whether there is any NA
crime%>%summarise_all(~ sum(is.na(.)))
```

```
##   M So Ed Po1 Po2 LF M.F Pop NW U1 U2 Wealth Ineq Prob Time Crime
## 1 0  0  0    0    0  0    0    0  0  0  0      0    0    0    0     0
```

From above, we can see there is no missing value.

Solution for Q10.1:

Build Regression Tree by rpart()

```
set.seed(73)
tree1<-rpart(Crime~.,data=crime,method = 'anova')
#method="anova" because we are building a regression tree, for classification tree, method="class"
tree1
```

```
## n= 47
##
## node), split, n, deviance, yval
##       * denotes terminal node
##
## 1) root 47 6880928.0  905.0851
##   2) Po1< 7.65 23  779243.5  669.6087
##     4) Pop< 22.5 12  243811.0  550.5000 *
##     5) Pop>=22.5 11  179470.7  799.5455 *
##   3) Po1>=7.65 24 3604162.0 1130.7500
##     6) NW< 7.65 10  557574.9  886.9000 *
##     7) NW>=7.65 14 2027225.0 1304.9290 *
```

The above is the tree's structure by indentation which is the same as the plot I put below.

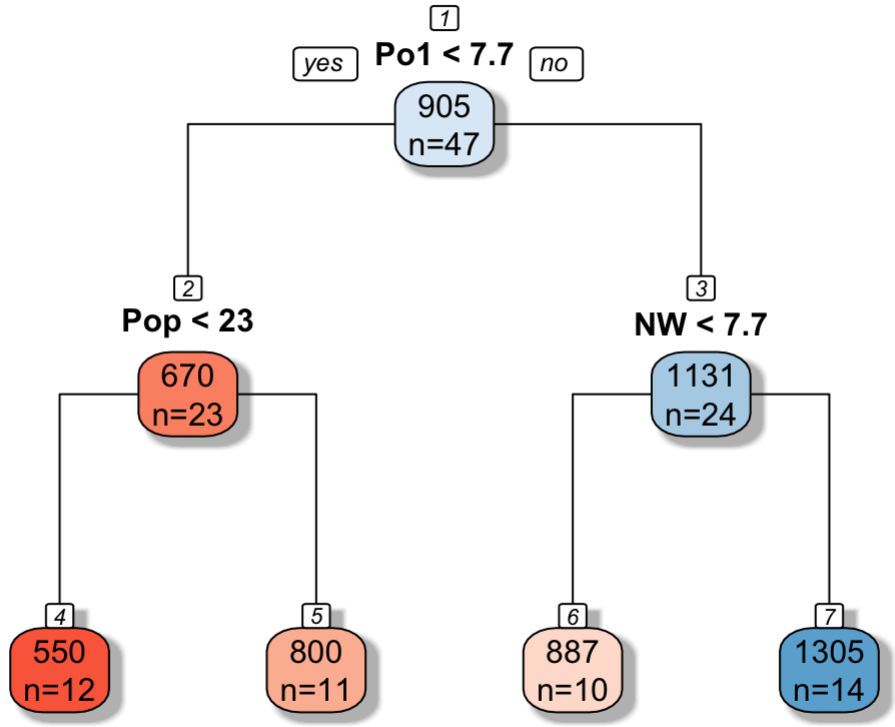
Some explanations about the output:

- node: The ith node in the tree. A smaller number means it is nearer to the root.
- split: The condition the split is based on.
- n: How many total samples are in this node.
- deviance: RSS=sum of squares of residuals=sum((every point-yval) ^2)
- yval: The prediction of each sample=mean(response value of all samples inside this node)

Note: the line marked with "*" at the end means it is a leaf.

Let's visualize the tree:

```
rpart.plot(tree1,box.palette="RdBu", type=1,shadow.col="gray", nn=TRUE,extra = 1)
```



From above we can tell the tree is split by only 3 predictors: Po1, Pop, and NW. There are 4 leaf nodes & 2 branching points.

We can see the first level of the split is divided by Po1, if Po1>=7.7, it belongs to the right node, and any sample with Po1<7.7 belongs to the left node. For lower-level splitting, the logic is the same. For each node, we can see there is a number on the top. This is the prediction of the response value of all data in this node. eg.n=14 for the most right low leaf means how many data points are in this node which =14.

Check the R_squared of our tree1 and double confirm whether yval=prediction for tree1

Since rpart already using 10 fold cross-validation, we can directly use the yval value to calculate the RSS(SSE) and R_squared.

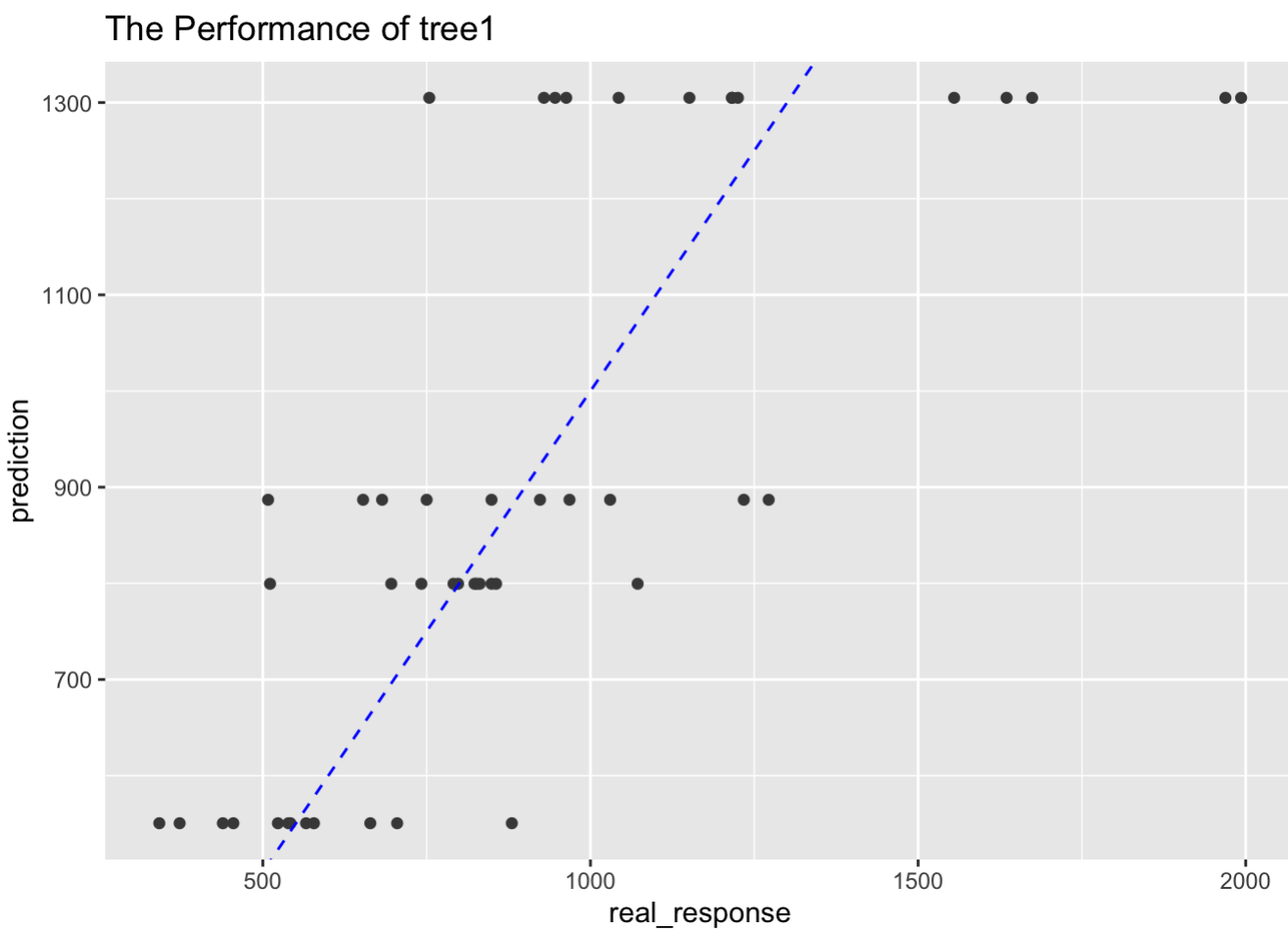
```
prediction_tree1<-predict(tree1)
head(prediction_tree1) # compare these value to the yhat above, they are the same.
```

```
##         1         2         3         4         5         6
##  799.5455 1304.9286  550.5000 1304.9286  886.9000  886.9000
```

```
#calculate the R^2
SSE<-sum((crime$Crime-prediction_tree1)^2) #sum of squared error
SST<-sum((crime$Crime-mean(crime$Crime))^2)

R_squared_tree1<-1-SSE/SST
R_squared_tree1
```

```
## [1] 0.5628378
```

Let's visualize the real response vs the predictions:



Since the regression tree predicts each data point by the mean value inside that node, we can see all the predictions are not spread around but plotted as horizontal dotted lines.

```
printcp(tree1)
```

```
##
## Regression tree:
## rpart(formula = Crime ~ ., data = crime, method = "anova")
##
## Variables actually used in tree construction:
## [1] NW  Po1 Pop
##
## Root node error: 6880928/47 = 146403
##
## n= 47
##
##         CP nsplit rel error  xerror    xstd
## 1 0.362963      0   1.00000 1.05104 0.25839
## 2 0.148143      1   0.63704 0.86717 0.18163
## 3 0.051732      2   0.48889 0.93600 0.23496
## 4 0.010000      3   0.43716 0.89390 0.23653
```

The cptable is the lower part of this output with columns: CP, nsplit, rel_error, xerror and xstd. These information are very important for pruning the tree.

- Root node error = The percent of incorrectly classification data point for classification tree
- Root node error = SST/n = MSE for regression tree

Pruning of the Regression Tree

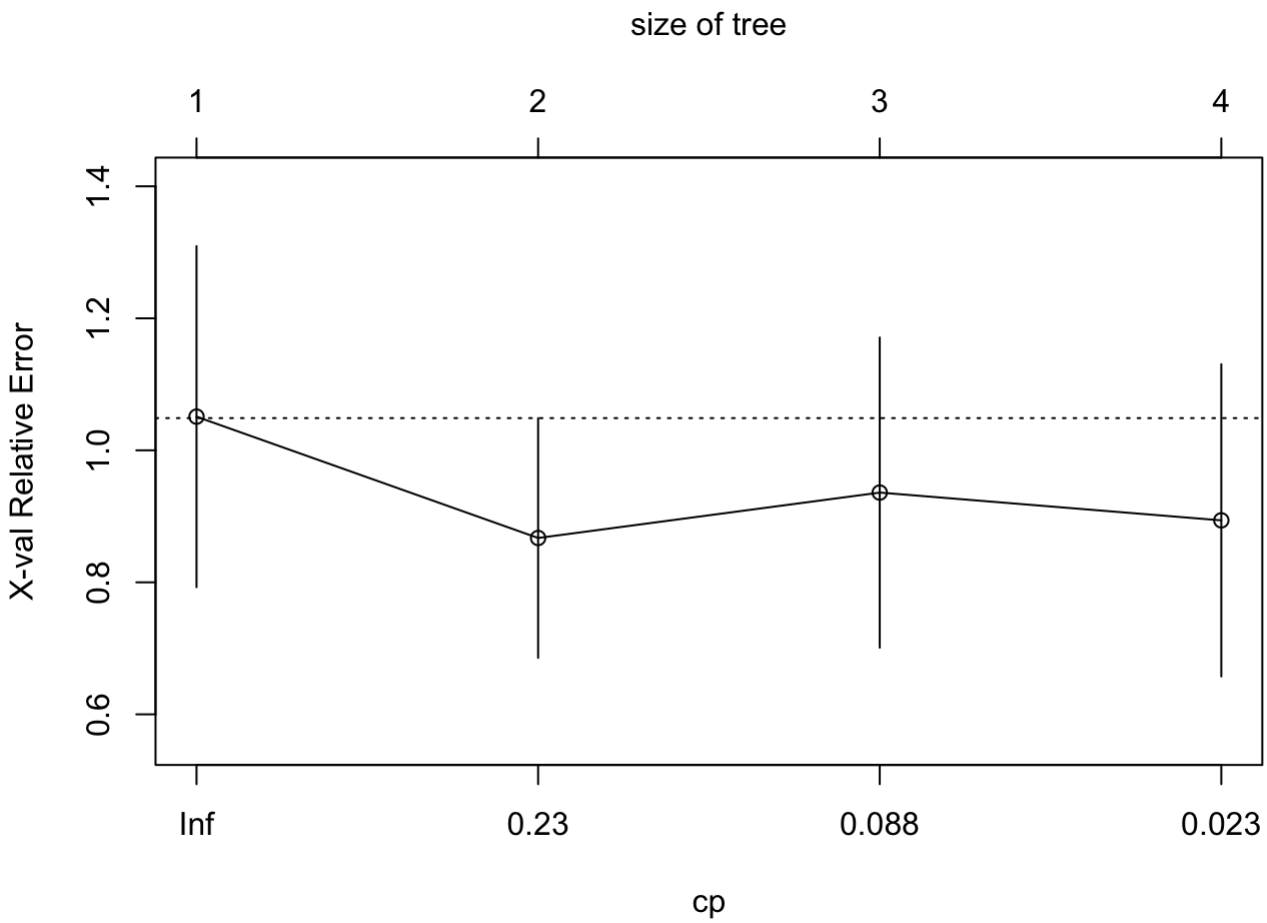
```
cptable<-data.frame(tree1$cptable)
cptable
```

```
##           CP nsplit rel.error    xerror      xstd
## 1 0.36296293      0 1.0000000 1.0510434 0.2583861
## 2 0.14814320      1 0.6370371 0.8671717 0.1816347
## 3 0.05173165      2 0.4888939 0.9359986 0.2349588
## 4 0.01000000      3 0.4371622 0.8938993 0.2365281
```

Some explanations for cptable:

- nsplit: the number of splits that has to be operated
- xerror: the percentage of cross-validation error compared with root total error (rpart has built-in cross-validation with 10 folds)
- rel_error: the percentage of real error compared with root total error from the training set
- xstd: standard error for xerror
- CP: complexity parameter, if any split does not increase the overall R_Squared of the model by at least a certain cp, the split is discarded. In rpart, the default cp=0.01. We can see above there is no split with CP <0.01.

```
plotcp(tree1) #check min xerror to decide where to prune
```



Two methods of how to decide where to prune the tree:

1. A simple rule would be to choose the tree with the smallest cross-validation error(xerror). But this tree might be complex. If we can find a simpler tree that performs as well as this tree, we can choose the simpler one.

2. 1-SE rule: if a tree's xerror is within 1 standard error of the min, they can be thought as equal. If one number is within one standard error from the other number, the two numbers can be thought of as statistically equal.

Step for 1-SE Rule:

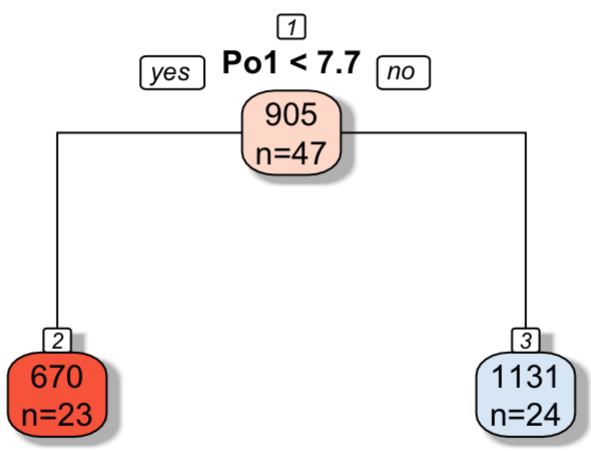
1. find min(xerror)
2. threshold=min(xerror)+ its xstd
3. find all xerror<threshold
4. use the smallest tree as our final tree

From the cptable, we can see min(xerror)=0.8671717, threshold=0.8671717+0.1816347=1.048806, the xerror of row 2,3,4 <1.0488, we then choose the smallest tree as the 2nd row with CP=0.14814320.

```
cpvalue <- cptable[which.min(cptable$xerror),1]
prune_tree1<-prune(tree1, cpvalue)
prune_tree1
```

```
## n= 47
##
## node), split, n, deviance, yval
##       * denotes terminal node
##
## 1) root 47 6880928.0  905.0851
##   2) Po1< 7.65 23  779243.5  669.6087 *
##   3) Po1>=7.65 24 3604162.0 1130.7500 *
```

```
rpart.plot(prune_tree1,box.palette="RdBu", type=1,shadow.col="gray", nn=TRUE,extra = 1)
```



Calculate R_squared of the prune_tree1 :

```
prediction_prunetree1<-predict(prune_tree1)
SSE_prunetree1<-sum((crime$Crime-prediction_prunetree1)^2)#sum of squared error

R_squared_prunetree1<-1-SSE_prunetree1/SST
R_squared_prunetree1
```

```
## [1] 0.3629629
```

The new R_squared of prune_tree1 is lower than the original tree1. This is not surprising. We built a more simple tree and tolerant more error to avoid overfitting.

Build Real Linear Regression Models with the 2 Leaves

rpart use the mean response value in each leaves as the prediction for all data inside that leaf. We can tell from the R_squared, the prediction is not very ideal. Let's try to use a real linear regression model on the split dataset and see whether the performance will be better.

```
prune_tree1$frame
```

```
##      var  n wt       dev      yval complexity ncompete nsurrogate
## 1    Po1 47 47 6880927.7  905.0851 0.36296293        4          5
## 2 <leaf> 23 23  779243.5  669.6087 0.05173165        0          0
## 3 <leaf> 24 24 3604162.5 1130.7500 0.14814320        0          0
```

```
prune_tree1$where #the number is the row index of prune_tree1$frame
```

```
##  [1] 2 3 2 3 3 3 3 3 2 2 3 2 2 2 2 3 2 3 3 3 2 2 3 3 2 3 2 3 3 2 2 3 2 3 3 3 2 2
## [39] 2 3 2 2 2 3 2 3 3
```

```
# 2 corresponding with Po1< 7.65, 3 corresponding with Po1>= 7.65

mask1<-prune_tree1$where==2
mask2<-prune_tree1$where==3
crime_sub1<-crime[mask1,] #will be used for lm model1
head(crime_sub1,4) # We can see all po1<7.65
```

```
##       M So   Ed Po1 Po2    LF   M.F Pop   NW    U1  U2 Wealth Ineq     Prob
## 1  15.1  1  9.1 5.8 5.6 0.510  95.0  33 30.1 0.108 4.1   3940 26.1 0.084602
## 3  14.2  1  8.9 4.5 4.4 0.533  96.9  18 21.9 0.094 3.3   3180 25.0 0.083401
## 9  15.7  1  9.0 6.5 6.2 0.553  95.5  39 28.6 0.081 2.8   4210 23.9 0.071697
## 10 14.0  0 11.8 7.1 6.8 0.632 102.9   7  1.5 0.100 2.4   5260 17.4 0.044498
##       Time Crime
## 1  26.2011   791
## 3  24.3006   578
## 9  29.4001   856
## 10 19.5994   705
```

```
crime_sub2<-crime[mask2,] #will be used for lm model2
```

Build linear regression model1 with the first leaf:

```
lm_model1<-lm(Crime~.,data=crime_sub1)
summary(lm_model1)
```

```
##
## Call:
## lm(formula = Crime ~ ., data = crime_sub1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -109.147  -52.803   -6.495   53.784  127.196
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)   -48.5477  2044.9766  -0.024   0.9817
## M              45.8622    58.6256   0.782   0.4597
## So            380.4815   223.1072   1.705   0.1319
## Ed            187.9074    89.5799   2.098   0.0741 .
## Po1            -3.5138   157.7513  -0.022   0.9829
## Po2            44.6382   148.5528   0.300   0.7725
## LF           1059.3652  1187.9722   0.892   0.4021
## M.F           -22.5521    21.4677  -1.051   0.3284
## Pop            10.6413     5.0929   2.089   0.0750 .
## NW              0.1010     7.9019   0.013   0.9902
## U1           4878.2802  4874.8165   1.001   0.3503
## U2             -5.5126   133.5094  -0.041   0.9682
## Wealth         -0.1022     0.1752  -0.583   0.5779
## Ineq            4.7779    35.5290   0.134   0.8968
## Prob        -7317.4407  3280.7511  -2.230   0.0609 .
## Time          -20.0603     7.7287  -2.596   0.0357 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 115.9 on 7 degrees of freedom
## Multiple R-squared:  0.8794, Adjusted R-squared:  0.6209
## F-statistic: 3.403 on 15 and 7 DF,  p-value: 0.0541
```

The R-squared and the adjusted R-squared are better than both of our tree models. But only 4 predictors are significant and the F-statistic with p-value>0.05 which means our model is not significant!

Let's eliminate the irrelevant predictors and build another linear regression model only with the significant predictors:(Making variable selection with coefficients' p-value is not ideal, but since we have such a high ratio with data amount to the number of the predictors together with the poor performance of the current model, it worth a try.)

```
lm_model1_4<-lm(Crime~Ed+Pop+Prob+Time,data=crime_sub1)
summary(lm_model1_4)
```

```
##
## Call:
## lm(formula = Crime ~ Ed + Pop + Prob + Time, data = crime_sub1)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -206.35  -90.22   -7.59   59.64  357.11
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   819.960    515.112   1.592   0.1288
## Ed              9.499     34.869   0.272   0.7884
## Pop            11.395      3.229   3.529   0.0024 **
## Prob        -3164.075   2095.755  -1.510   0.1485
```

```
## Time          -12.130      6.830  -1.776   0.0927 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 154.5 on 18 degrees of freedom
## Multiple R-squared:  0.4485, Adjusted R-squared:  0.3259
## F-statistic: 3.659 on 4 and 18 DF,  p-value: 0.02379
```

This model is better with F-statistic but only Pop and Time are significant. Both R-squared and Adjusted R-squared decrease by around 0.2. I can't see any improvement at this point.

Let's final try with only Time+Pop as our predictors:

```
lm_model1_2<-lm(Crime~Pop+Time,data=crime_sub1)
summary(lm_model1_2)
```

```
##
## Call:
## lm(formula = Crime ~ Pop + Time, data = crime_sub1)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -185.42 -117.40  -23.00   36.78  391.19
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  638.392    142.637   4.476 0.000232 ***
## Pop            9.571      3.114   3.074 0.005993 **
## Time          -6.969      6.255  -1.114 0.278412
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 160.7 on 20 degrees of freedom
## Multiple R-squared:  0.3369, Adjusted R-squared:  0.2706
## F-statistic: 5.081 on 2 and 20 DF,  p-value: 0.01644
```

The F-statistic is better with the p-value of 0.0164 and only Pop is significant. The Adjusted r-squared turns to be worse as 0.271. I still can't see any great improvement at this point. With such a small dataset, it is really hard to get a nice linear regression model.

Build linear regression model1 with the 2nd leaf:

```
lm_model2<-lm(Crime~.,data=crime_sub2)
summary(lm_model2)
```

```
##
## Call:
## lm(formula = Crime ~ ., data = crime_sub2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -206.805 -120.407   -9.489  103.985  248.226
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -8634.1701  2366.4043  -3.649  0.00651 **
## M               5.6032    96.1623   0.058  0.95496
## So            179.6267   409.5210   0.439  0.67254
## Ed            263.0845   146.4229   1.797  0.11010
## Po1           235.2349   166.1289   1.416  0.19452
## Po2          -140.7023   193.8759  -0.726  0.48869
```

```
## LF           1442.4214  4832.4463   0.298  0.77294
## M.F            -1.2379    54.8160  -0.023  0.98254
## Pop            -3.7686     2.8833  -1.307  0.22751
## NW             -0.5396    24.5039  -0.022  0.98297
## U1          -3779.9843 10923.3434  -0.346  0.73823
## U2            163.7106   150.5361   1.088  0.30848
## Wealth          0.3017     0.2051   1.471  0.17946
## Ineq          155.3754    65.5077   2.372  0.04511 *
## Prob        -3624.0711  4381.4724  -0.827  0.43214
## Time           21.9335    14.6754   1.495  0.17338
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 229.9 on 8 degrees of freedom
## Multiple R-squared:  0.8827, Adjusted R-squared:  0.6626
## F-statistic: 4.012 on 15 and 8 DF,  p-value: 0.02669
```

Only Ineq and the Intercept is significant. The adjusted R-squared and R-squared are good. The F-statistic is still significant which is good. However, with so many predictors irrelevant, we can't just use this model. So let's try to eliminate all predictors but only keep Ineq as our predictor!

```
lm_model2_1<-lm(Crime~Ineq,data=crime_sub2)
summary(lm_model2_1)
```

```
##
## Call:
## lm(formula = Crime ~ Ineq, data = crime_sub2)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -612.67 -254.25  -88.83  136.25  918.77
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   702.70     495.52   1.418    0.170
## Ineq           24.44      27.91   0.876    0.391
##
## Residual standard error: 397.9 on 22 degrees of freedom
## Multiple R-squared:  0.03368,    Adjusted R-squared:  -0.01024
## F-statistic: 0.7668 on 1 and 22 DF,  p-value: 0.3907
```

Neither the Ineq nor the intercept is significant and the F-statistics' p-value is so big means that the model doesn't show any value. In my opinion, the 2nd leaf of the data is not proper for the linear regression model. We will need to try to use other models to fit it in the future.

Here is the table for comparing all models:

Model	R.squared..Adjusted.for.lm.model.	F.statistic.p.value	Predictors.Use
tree1	0.56280	NA	All
pruntree1	0.36300	NA	All
lm_model1	0.62090	0.05410	All
lm_model1_4	0.32590	0.02379	4
lm_model1_2	0.27060	0.01644	2
lm_model2	0.66260	0.02669	All

lm_model2_1	-0.01024	0.39070	1

With trying so many linear regression models, all of the results are not that ideal. Only the model generated by the 2nd leaf with all predictors seems better (but the p-value for F-statistic is at the edge of statistical significance). Without cross-validation, we can't guarantee its performance on other real data in the future. However, cross-validation with such small training data (23-24 data points on each leaf) is also meaningless.

RandomForests Model

```
set.seed(73)
forest<-randomForest(formula=Crime~.,data=crime,replace=TRUE,importance=T,keep.forest=TRUE)
forest
```

```
##
## Call:
##  randomForest(formula = Crime ~ ., data = crime, replace = TRUE,      importance = T, keep.forest
= TRUE)
##                Type of random forest: regression
##                      Number of trees: 500
## No. of variables tried at each split: 5
##
##           Mean of squared residuals: 84422.28
##                     % Var explained: 42.34
```

- % Var explained is the R_squared value.
- ntree=500: build 500 trees is the default, I didn't specify another number here because of our super small dataset.
- mtry: how many random predictors to choose from for the split. The default number for regression tree=n/3, classification tree=sqrt(n), n=how many predictors we have
- replace=True: when sample the original data, we can duplicate the data points.

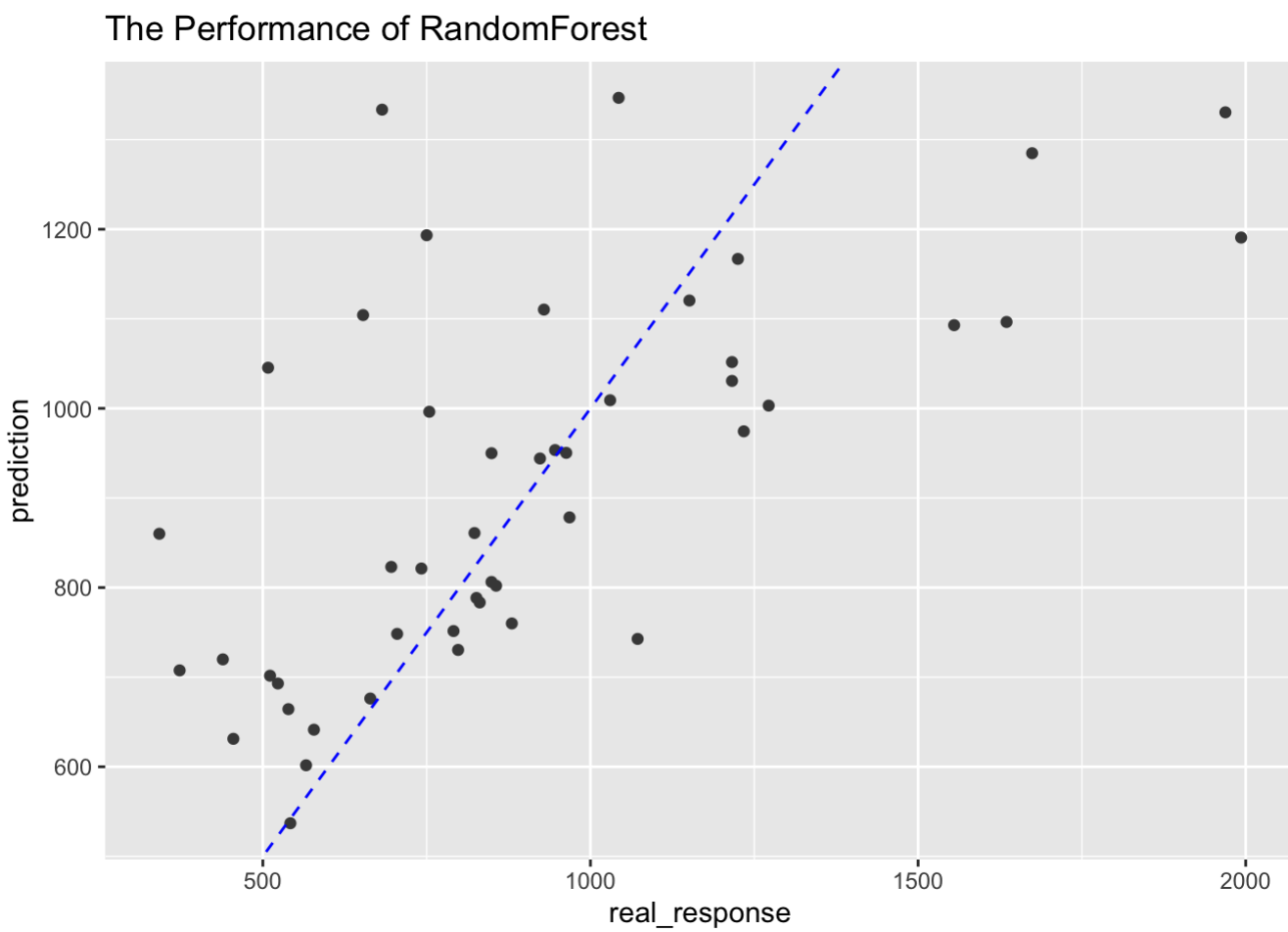
Note: increasing the number of trees(mtry) to high values does not necessarily make the model overfit the data due to the averaging involved. Usually increasing the number of trees will decrease the test error until a certain threshold is reached. Once this threshold is reached increasing the number of trees will just keep the performance at this threshold. Our goal is to find a number of trees that is sufficiently large to reach an acceptable test error threshold but not too large that costs unnecessary computation.

For our situation, with such as small dataset (only 47 states), 500 trees are quite enough so I won't test any other larger number of trees for my random forest model.

```
#Manually calculate R_squared to check whether =%Var explained
R_squared_forest<-1-sum((forest$predicted-crime$Crime)^2)/SST
R_squared_forest
```

```
## [1] 0.4233558
```

From above we can see they are equal. Furthermore, this R_squared < decision tree1's R_squared which equal to 0.5628. It is not surprising since randomForest is good for avoiding overfitting.



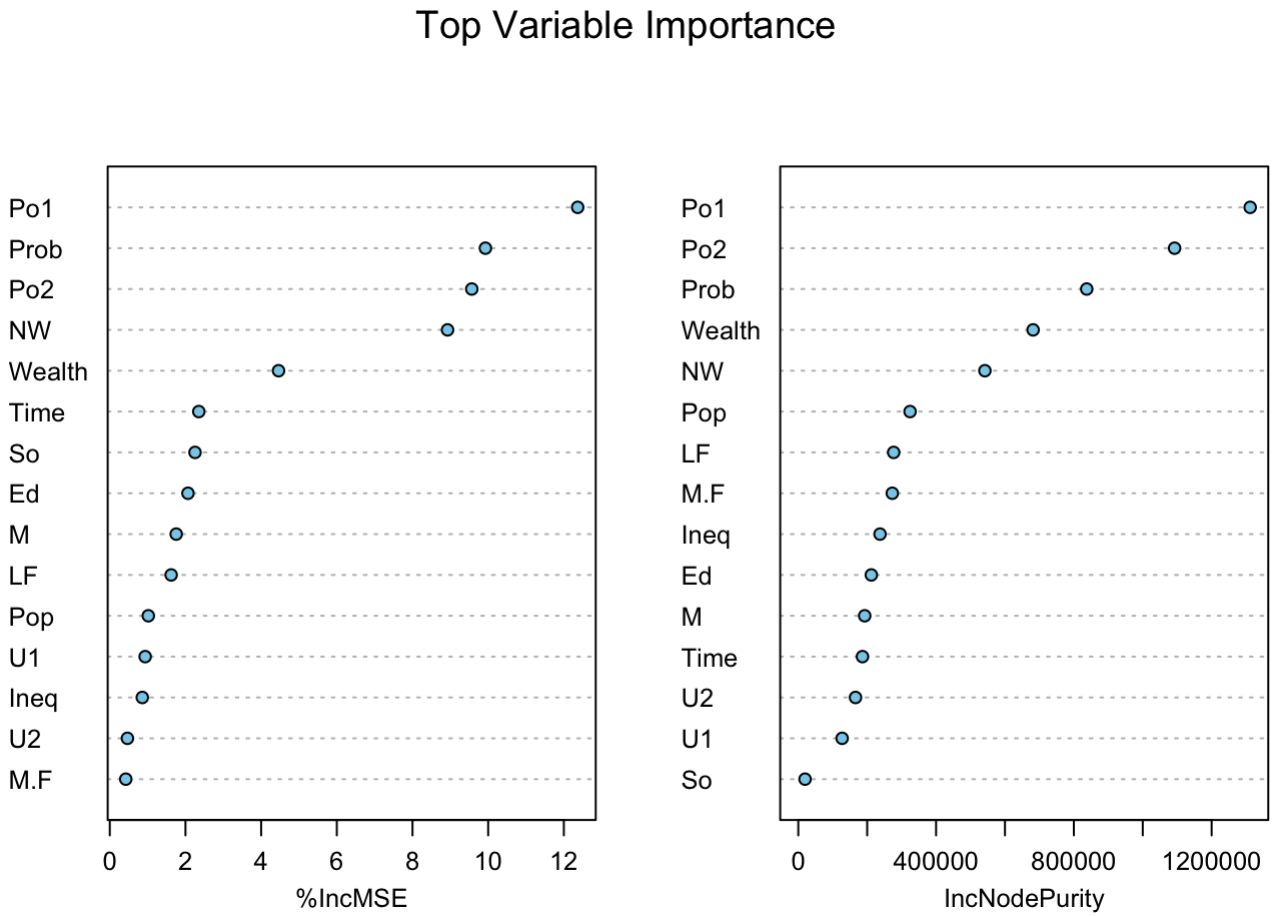
Just compare the predictions from the random forest vs regression decision tree. The predictions are more spread out and look like it makes more sense since all of the values are the average of these 500 trees. The overfitting is much more reduced!

Random forests can be used to rank the importance of variables in a regression or classification problem. Let's check the importance of our predictors:

```
importance(forest)
```

```
##           %IncMSE IncNodePurity
## M       1.7553006     192749.35
## So      2.2524958      19589.36
## Ed      2.0667656     212018.29
## Po1    12.3653589    1311946.80
## Po2     9.5670769    1092477.72
## LF      1.6221096     276908.54
## M.F     0.4221449     272867.55
## Pop     1.0179300     324474.52
## NW      8.9266946     541708.45
## U1      0.9351193     127260.58
## U2      0.4657518     166091.81
## Wealth  4.4599943     681722.46
## Ineq    0.8592246     237288.77
## Prob    9.9271663     837336.96
## Time    2.3524305     186329.70
```

```
varImpPlot(forest,main = 'Top Variable Importance',bg = "skyblue",cex=0.8)
```



- %IncMSE: Percentage increase in mean square error is analogous to accuracy-based importance, and is calculated by shuffling the values of the out-of-bag samples.
- IncNodePurity: Increase in node purity is analogous to Gini-based importance, and is calculated based on the reduction in the sum of squared errors whenever a variable is chosen to split.

%IncMSE and IncNodePurity are both good indicators for variable importance. But their ranks are not always the same. (Take our plot as an example, the Wealth and NW's rankings are different.)

From the plot we can tell the most important predictor for splitting is Po1 which is the same as our decision tree analysis.

Manually Cross-Validation with RandomForests Model

Normally speaking, it is not necessary to run cross-validation with random forest because it already runs so many trees randomly and is fair enough to prevent overfitting. If we have a large dataset, we can expect to see the performance of cv is equal to the random forest built directly from the dataset.

However, let's experiment with the cross-validation here on our super tiny data. I aim to check whether the performance from cv is still equal to the performance of our random forest model–forest.

Split the crime dataset into 6 folders and run cross-validation to check the model's real performance:

```
set.seed(73)
train_validation_data<-crime
train_validation_data$label<-sample(factor(rep(1:6, length.out = 47))) # 6 folders
head(train_validation_data,3)
```

```
##      M So   Ed  Po1 Po2    LF   M.F Pop   NW    U1  U2 Wealth Ineq     Prob
## 1 15.1  1  9.1  5.8 5.6 0.510  95.0  33 30.1 0.108 4.1   3940 26.1 0.084602
```

```
## 2 14.3  0 11.3 10.3 9.5 0.583 101.2  13 10.2 0.096 3.6   5570 19.4 0.029599
## 3 14.2  1  8.9  4.5 4.4 0.533  96.9  18 21.9 0.094 3.3   3180 25.0 0.083401
##      Time Crime label
## 1 26.2011   791     1
## 2 25.2999  1635     6
## 3 24.3006   578     5
```

```
#create df_lst contain 6 sub_dfs
label<-1:6
df_list<-list()
for (i in label){
  df_temp<-train_validation_data[train_validation_data$label==i,]
  df_list[[i]]<-df_temp
}

R_squared<-c()
for (i in label){
  validation_response<-df_list[[i]][,16] #Crime columns as real response
  validation<-df_list[[i]][,-c(17,16)] #eliminate label and Crime column
  train<-bind_rows(df_list[-i])[,-17] #eliminate label column
  set.seed(73)
  model<-randomForest(Crime~.,data=train,importance=T)
  predictions<- predict(model,newdata=validation)
  sst<-sum((validation_response-mean(validation_response))^2)
  r2<-1-sum((validation_response-predictions)^2)/sst
  R_squared<-c(R_squared,r2)
}
R_squared
```

```
## [1]  0.42258537  0.04682513 -0.40353922  0.50513480  0.65201047  0.09964673
```

```
mean(R_squared)
```

```
## [1] 0.2204439
```

The R_squared from cross-validation is much lower than the original forest model(0.423). Most of the time, the performance from cross-validation should be lower than the one with the training set since it didn't count in the fit of the random pattern. But random forest should have already spread out the random pattern in many trees and provide a much more unbiased result. This lower score here is mainly because of our super small dataset. With split into 6 folds, some forest built by 5 of the folds doesn't have a chance to learn the important data points in the 6th fold which may cause a big error. From the R_squared vector, we can see the R_squared are very unstable, even negative values show up.

Build 2 RandomForests Models with 2 different leaves

```
set.seed(73)
forest_sub1<-randomForest(formula=Crime~.,data=crime_sub1,replace=TRUE,importance=T,keep.forest=TRUE)
forest_sub1
```

```
##
## Call:
##  randomForest(formula = Crime ~ ., data = crime_sub1, replace = TRUE,      importance = T, keep.fo
rest = TRUE)
##                Type of random forest: regression
##                      Number of trees: 500
## No. of variables tried at each split: 5
##
```

```
##           Mean of squared residuals: 28473.62
##                     % Var explained: 15.96
```

With the leaf that Po1<7.65, our R-squared drop to 16% which is caused by an even smaller dataset (23 data points).

```
importance(forest_sub1)
```

```
##           %IncMSE IncNodePurity
## M       9.3743827     98244.128
## So      2.1270144      7850.913
## Ed      4.4813225     32027.309
## Po1     5.5604154     70487.764
## Po2     4.1892551     46864.435
## LF      0.2639056     25396.532
## M.F    -0.7351965     21314.920
## Pop     6.1270326    138236.089
## NW      5.6798947     77046.088
## U1     -0.2597235     36471.759
## U2     -2.8338067     31341.538
## Wealth  1.9981343     23324.562
## Ineq    0.6361782     29666.617
## Prob   -2.1749912     33801.180
## Time    3.2146206     35269.107
```

Very interesting found here is after we split the dataset by Po1, the most important variable in leaf 1 is not Po1 anymore but change to M! This is reasonable since we already use Po1 to split our data, the variable in leaf 1 that affects most of our prediction error is changed to another predictor.

```
set.seed(73)
forest_sub2<-randomForest(formula=Crime~.,data=crime_sub2,replace=TRUE,importance=T,keep.forest=TRUE)
forest_sub2
```

```
##
## Call:
##  randomForest(formula = Crime ~ ., data = crime_sub2, replace = TRUE,      importance = T, keep.fo
rest = TRUE)
##                Type of random forest: regression
##                      Number of trees: 500
## No. of variables tried at each split: 5
##
##           Mean of squared residuals: 143370
##                     % Var explained: 4.53
```

We can see the R-squared drops to 4.53% which means the random forest model is not ideal for the 2nd leaf(Po1>=7.65). Till now, we haven't found a good model to use on either side of our leaves. Neither linear regression nor random forest. With such a small dataset, it is indeed hard!

```
importance(forest_sub2)
```

```
##           %IncMSE IncNodePurity
## M       4.7823226    273907.288
## So      0.2174256      4036.154
## Ed     -1.5781929    130461.678
## Po1     3.1440802    330199.195
## Po2     2.5804093    279298.835
## LF     -1.1983135    206736.658
## M.F    -0.4048299    230974.897
```

```
## Pop    -1.1986214    122994.159
## NW      4.7682690    265718.520
## U1     -0.8015800    165509.163
## U2      1.7919121     94354.954
## Wealth  2.6052702    398188.409
## Ineq    1.1394657    190220.736
## Prob    3.8222800    316873.167
## Time    0.9447560    162885.161
```

The most important variables in leaf 2 are M and NW.

Question 10.2

Describe a situation or problem from your job, everyday life, current events, etc., for which a logistic regression model would be appropriate. List some (up to 5) predictors that you might use.

Solution for Q10.2:

The real estate market is red hot since the pandemic. Most of the time, people purchase houses with a loan from banks. Since the house is not 100% owned by the new homeowner, the bank usually requires the new homeowner to buy a homeowner's insurance.

However, owning a house and having the ability to pay the fee for the homeowner's insurance can't guarantee the owner is eligible for the insurance. To help an insurance company decide whether to provide insurance to the applicant, here are 5 predictors, from which the insurance company can get a probability of whether the candidate is a good one or not. By setting a threshold, the company can easily decide whether to approve the application or not.

- The year the home was built (numeric)
- The value of the house (numeric)
- Whether the home is located at hazard area (flood zone, wildfire zone, etc) (binary)
- Whether the applicant has homeowners' insurance in the past (binary)
- The applicant's credit score (numeric)

Question 10.3

1. Using the GermanCredit data set germancredit.txt from http://archive.ics.uci.edu/ml/machine- learning-databases/statlog/german / (description at http://archive.ics.uci.edu/ml/datasets/Statlog+%28German+Credit+Data%29), use logistic regression to find a good predictive model for whether credit applicants are good credit risks or not. Show your model (factors used and their coefficients), the software output, and the quality of fit. You can use the glm function in R. To get a logistic regression (logit) model on data where the response is either zero or one, use family=binomial(link="logit") in your glm function call.

2. Because the model gives a result between 0 and 1, it requires setting a threshold probability to separate between "good" and "bad" answers. In this data set, they estimate that incorrectly identifying a bad customer as good, is 5 times worse than incorrectly classifying a good customer as bad. Determine a good threshold probability based on your model.

General Observation

```
credit<-read.delim('germancredit.txt',stringsAsFactors=TRUE,header = F,sep=' ')
#data description
head(credit,3)
```

```
##    V1 V2  V3  V4   V5  V6  V7 V8  V9  V10 V11  V12 V13  V14  V15 V16  V17 V18
## 1 A11  6 A34 A43 1169 A65 A75  4 A93 A101   4 A121  67 A143 A152   2 A173   1
## 2 A12 48 A32 A43 5951 A61 A73  2 A92 A101   2 A121  22 A143 A152   1 A173   1
## 3 A14 12 A34 A46 2096 A61 A74  2 A93 A101   3 A121  49 A143 A152   1 A172   2
##    V19  V20 V21
```

```
## 1 A192 A201   1
## 2 A191 A201   2
## 3 A191 A201   1
```

Here are the description of all variables:

Attribute 1: (qualitative)

Status of existing checking account

A11 : ... < 0 DM

A12 : 0 <= ... < 200 DM

A13 : ... >= 200 DM / salary assignments for at least 1 year

A14 : no checking account

Attribute 2: (numerical)

Duration in month

Attribute 3: (qualitative)

Credit history

A30 : no credits taken/ all credits paid back duly

A31 : all credits at this bank paid back duly

A32 : existing credits paid back duly till now

A33 : delay in paying off in the past

A34 : critical account/ other credits existing (not at this bank)

Attribute 4: (qualitative)

Purpose

A40 : car (new)

A41 : car (used)

A42 : furniture/equipment

A43 : radio/television

A44 : domestic appliances

A45 : repairs

A46 : education

A47 : (vacation - does not exist?)

A48 : retraining

A49 : business

A410 : others

Attribute 5: (numerical)

Credit amount

Attibute 6: (qualitative)

Savings account/bonds

A61 : ... < 100 DM

A62 : 100 <= ... < 500 DM

A63 : 500 <= ... < 1000 DM

A64 : .. >= 1000 DM

A65 : unknown/ no savings account

Attribute 7: (qualitative)

Present employment since

A71 : unemployed

A72 : ... < 1 year

A73 : 1 <= ... < 4 years

A74 : 4 <= ... < 7 years

A75 : .. >= 7 years

Attribute 8: (numerical)

Installment rate in percentage of disposable income

Attribute 9: (qualitative)

Personal status and sex

A91 : male : divorced/separated

A92 : female : divorced/separated/ married

A93 : male : single

A94 : male : married/widowed

A95 : female : single

Attribute 10: (qualitative)

Other debtors / guarantors

A101 : none

A102 : co-applicant

A103 : guarantor

Attribute 11: (numerical)

Present residence since

Attribute 12: (qualitative)

Property

A121 : real estate

A122 : if not A121 : building society savings agreement/ life insurance

A123 : if not A121/A122 : car or other, not in attribute 6

A124 : unknown / no property

Attribute 13: (numerical)

Age in years

Attribute 14: (qualitative)

Other installment plans

A141 : bank

A142 : stores

A143 : none

Attribute 15: (qualitative)

Housing

A151 : rent

A152 : own

A153 : for free

Attribute 16: (numerical)

Number of existing credits at this bank

Attribute 17: (qualitative)

Job

A171 : unemployed/ unskilled - non-resident

A172 : unskilled - resident

A173 : skilled employee / official

A174 : management/ self-employed/ highly qualified employee/ officer

Attribute 18: (numerical)

Number of people being liable to provide maintenance for

Attribute 19: (qualitative)

Telephone

A191 : none

A192 : yes, registered under the customers name

Attribute 20: (qualitative)

foreign worker

A201 : yes

A202 : no

```
str(credit)
```

```
## 'data.frame':    1000 obs. of  21 variables:
##  $ V1 : Factor w/ 4 levels "A11","A12","A13",..: 1 2 4 1 1 4 4 2 4 2 ...
##  $ V2 : int  6 48 12 42 24 36 24 36 12 30 ...
##  $ V3 : Factor w/ 5 levels "A30","A31","A32",..: 5 3 5 3 4 3 3 3 3 5 ...
##  $ V4 : Factor w/ 10 levels "A40","A41","A410",..: 5 5 8 4 1 8 4 2 5 1 ...
##  $ V5 : int  1169 5951 2096 7882 4870 9055 2835 6948 3059 5234 ...
##  $ V6 : Factor w/ 5 levels "A61","A62","A63",..: 5 1 1 1 1 5 3 1 4 1 ...
##  $ V7 : Factor w/ 5 levels "A71","A72","A73",..: 5 3 4 4 3 3 5 3 4 1 ...
##  $ V8 : int  4 2 2 2 3 2 3 2 2 4 ...
##  $ V9 : Factor w/ 4 levels "A91","A92","A93",..: 3 2 3 3 3 3 3 3 1 4 ...
##  $ V10: Factor w/ 3 levels "A101","A102",..: 1 1 1 3 1 1 1 1 1 1 ...
##  $ V11: int  4 2 3 4 4 4 4 2 4 2 ...
##  $ V12: Factor w/ 4 levels "A121","A122",..: 1 1 1 2 4 4 2 3 1 3 ...
##  $ V13: int  67 22 49 45 53 35 53 35 61 28 ...
##  $ V14: Factor w/ 3 levels "A141","A142",..: 3 3 3 3 3 3 3 3 3 3 ...
##  $ V15: Factor w/ 3 levels "A151","A152",..: 2 2 2 3 3 3 2 1 2 2 ...
##  $ V16: int  2 1 1 1 2 1 1 1 1 2 ...
```

```
##  $ V17: Factor w/ 4 levels "A171","A172",..: 3 3 2 3 3 2 3 4 2 4 ...
##  $ V18: int  1 1 2 2 2 2 1 1 1 1 ...
##  $ V19: Factor w/ 2 levels "A191","A192": 2 1 1 1 1 2 1 2 1 1 ...
##  $ V20: Factor w/ 2 levels "A201","A202": 1 1 1 1 1 1 1 1 1 1 ...
##  $ V21: int  1 2 1 1 2 1 1 1 1 2 ...
```

From above we can see there are 1000 data points in the credit dataset with 21 columns. 7 of them are numerical, 13 are categorical. The last one V21 is our response column with values 1 and 2. We will need to change them to 0 and 1 for our logistic regression model.

Note 1: For logistic regression, we don't need to scale our data. Scaling might help speed up gradient descent. This is because θ will descend quickly on small ranges and slowly on large ranges, and so will oscillate inefficiently down to the optimum when the variables are very uneven.

Note 2: Many columns are categorical. With this kind of data, R automatically uses one-hot encoding to interpret each level in all categorical predictors as a new predictor for the final response. The value for the new predictor is either 0 or 1. 0 indicates non-existent while 1 indicates existent. From below you can see the output from the logistic regression model with the coefficients of V1A12, V1A13, etc.

Preprocess the response column:

```
credit$V21[credit$V21==1]<-0
credit$V21[credit$V21==2]<-1
```

Separate the dataset into 80% training and 20% testing:

```
set.seed(73)
train<-sample_n(credit,size=800,replace=F)
test<-credit%>%setdiff(train)
```

```
#check whether there is any overlap between training and test dataset
train%>%inner_join(test)
```

```
##  [1] V1  V2  V3  V4  V5  V6  V7  V8  V9  V10 V11 V12 V13 V14 V15 V16 V17 V18 V19
## [20] V20 V21
## <0 rows> (or 0-length row.names)
```

We can see it is split successfully.

Logistic Regression Model with All Predictors

```
log_model1<-glm(V21~.,family=binomial(link ='logit'),data=train)
summary(log_model1)
```

```
##
## Call:
## glm(formula = V21 ~ ., family = binomial(link = "logit"), data = train)
##
## Deviance Residuals:
##     Min       1Q   Median       3Q      Max
## -2.1973  -0.6976  -0.3683   0.6941   2.9700
##
## Coefficients:
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept)  8.833e-01  1.212e+00   0.729 0.466276
## V1A12       -2.274e-01  2.432e-01  -0.935 0.349621
## V1A13       -7.204e-01  4.156e-01  -1.733 0.083014 .
```

```
## V1A14       -1.754e+00  2.619e-01  -6.698 2.11e-11 ***
## V2           2.697e-02  1.030e-02   2.617 0.008860 **
## V3A31       -3.486e-01  6.341e-01  -0.550 0.582489
## V3A32       -9.596e-01  5.014e-01  -1.914 0.055632 .
## V3A33       -1.208e+00  5.569e-01  -2.170 0.030006 *
## V3A34       -1.714e+00  5.174e-01  -3.313 0.000923 ***
## V4A41       -1.492e+00  4.107e-01  -3.632 0.000282 ***
## V4A410      -1.415e+00  8.263e-01  -1.713 0.086744 .
## V4A42       -8.906e-01  2.933e-01  -3.036 0.002394 **
## V4A43       -1.077e+00  2.785e-01  -3.867 0.000110 ***
## V4A44       -1.373e+00  9.414e-01  -1.459 0.144599
## V4A45       -9.927e-01  6.401e-01  -1.551 0.120963
## V4A46        1.633e-01  4.257e-01   0.384 0.701282
## V4A48       -1.550e+01  4.811e+02  -0.032 0.974301
## V4A49       -1.265e+00  3.908e-01  -3.236 0.001211 **
## V5           1.042e-04  4.913e-05   2.120 0.034000 *
## V6A62       -2.545e-01  3.358e-01  -0.758 0.448387
## V6A63       -4.501e-01  4.751e-01  -0.947 0.343431
## V6A64       -1.654e+00  6.866e-01  -2.409 0.015989 *
## V6A65       -8.948e-01  2.886e-01  -3.100 0.001932 **
## V7A72       -3.850e-02  4.995e-01  -0.077 0.938566
## V7A73       -7.708e-02  4.848e-01  -0.159 0.873672
## V7A74       -6.074e-01  5.149e-01  -1.180 0.238143
## V7A75        4.141e-02  4.879e-01   0.085 0.932371
## V8           3.041e-01  1.004e-01   3.030 0.002448 **
## V9A92       -1.972e-01  4.362e-01  -0.452 0.651201
## V9A93       -7.623e-01  4.318e-01  -1.765 0.077488 .
## V9A94       -4.794e-01  5.212e-01  -0.920 0.357654
## V10A102      7.471e-01  4.589e-01   1.628 0.103515
## V10A103     -8.587e-01  4.714e-01  -1.821 0.068540 .
## V11         -3.015e-02  9.792e-02  -0.308 0.758135
## V12A122      4.103e-01  2.793e-01   1.469 0.141923
## V12A123      1.775e-01  2.650e-01   0.670 0.502875
## V12A124      5.287e-01  4.655e-01   1.136 0.256017
## V13         -2.515e-02  1.065e-02  -2.363 0.018149 *
## V14A142     -5.498e-02  4.741e-01  -0.116 0.907666
## V14A143     -6.988e-01  2.680e-01  -2.607 0.009126 **
## V15A152     -1.648e-01  2.725e-01  -0.605 0.545283
## V15A153     -4.445e-01  5.304e-01  -0.838 0.401978
## V16          2.869e-01  2.124e-01   1.351 0.176709
## V17A172      6.882e-01  7.731e-01   0.890 0.373319
## V17A173      7.868e-01  7.444e-01   1.057 0.290492
## V17A174      5.229e-01  7.536e-01   0.694 0.487750
## V18          1.628e-01  2.867e-01   0.568 0.570241
## V19A192     -1.874e-01  2.272e-01  -0.825 0.409508
## V20A202     -9.989e-01  6.691e-01  -1.493 0.135436
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 967.00  on 799  degrees of freedom
## Residual deviance: 711.27  on 751  degrees of freedom
## AIC: 809.27
##
## Number of Fisher Scoring iterations: 14
```

We can see all the categorical predictors' levels become individual predictors and have their own coefficients. Some of them are statistical significant, some are not. I will do some variable selection based on the p-value later.

Predict Our Test Data:

```
thres=0.5
predict1<-predict(log_model1,test,type="response")
#need to set type="response" or the output is log-odds and hard to read
head(predict1)
```

```
##          1          2          3          4          5          6
## 0.06250902 0.78625591 0.25266128 0.47545696 0.55636678 0.06751165
```

```
predict1_binary<-as.integer(predict1>thres)
head(predict1_binary)
```

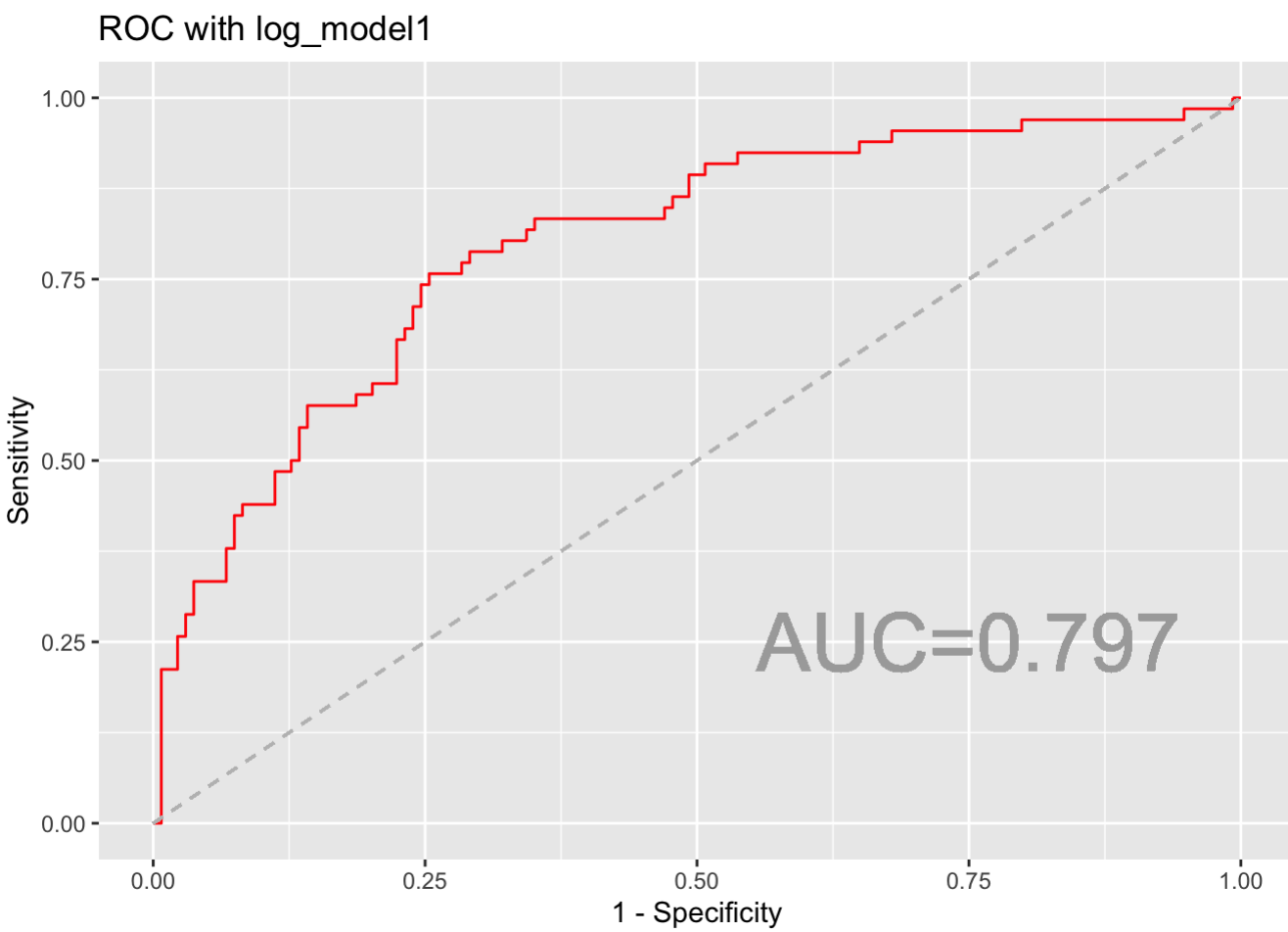
```
## [1] 0 1 0 0 1 0
```

Calculate ROC and AUC for log_model1

```
roc_log_model1<-roc(test$V21,predict1)
roc_log_model1
```

```
##
## Call:
## roc.default(response = test$V21, predictor = predict1)
##
## Data: predict1 in 134 controls (test$V21 0) < 66 cases (test$V21 1).
## Area under the curve: 0.7974
```

Draw ROC Curve:



Form the confusion matrix:

```
confuse_logmodel1_0.5<-as.matrix(table(predict1_binary,test$V21))
dimnames(confuse_logmodel1_0.5) = list(Predicted = c("good=0", "bad=1"), Actual= c("good=0", "bad=1"
))
confuse_logmodel1_0.5
```

```
##          Actual
## Predicted good=0 bad=1
##    good=0    119    36
##    bad=1      15    30
```

```
# the column is the real response, the row is predicted. Be careful this is different from our modul
e.
# 0=Positive=good, 1=Negative=bad

sensiitivity_logmodel1_0.5<-confuse_logmodel1_0.5[1,1]/sum(confuse_logmodel1_0.5[,1])
sensiitivity_logmodel1_0.5
```

```
## [1] 0.8880597
```

```
specificity_logmodel1_0.5<-confuse_logmodel1_0.5[2,2]/sum(confuse_logmodel1_0.5[,2])
specificity_logmodel1_0.5
```

```
## [1] 0.4545455
```

```
accuracy_logmodel1_0.5<-sum(diag(confuse_logmodel1_0.5))/sum(confuse_logmodel1_0.5)
accuracy_logmodel1_0.5
```

```
## [1] 0.745
```

The accuracy=0.745, Sensitivity=0.888, Specificity=0.455 for log_model1 with threshold=0.5.

Which threshold is the best for our log_model1? It should result in the highest accuracy together with both relatively high sensitivity and specificity. Furthermore, the cost of FP is 5 times the cost of FN. (0 = Good = Positive, 1 = Bad = Negative) We should consider the cost with the accuracy together for our final logistic regression model!

Total cost of miss identify=5FP+FN

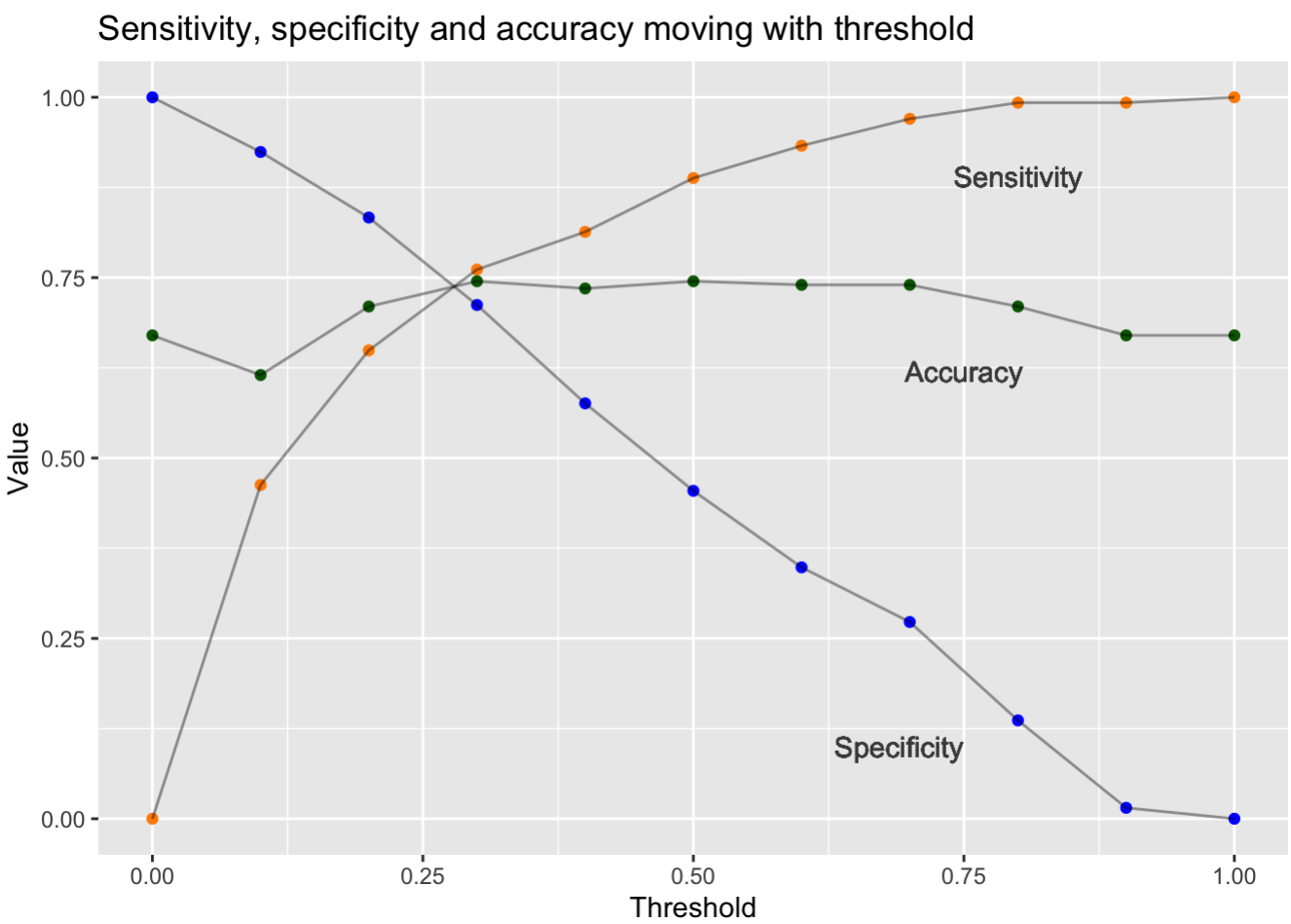
Let's run a loop for threshold starts from 0.1 to 0.9 with step 0.1:

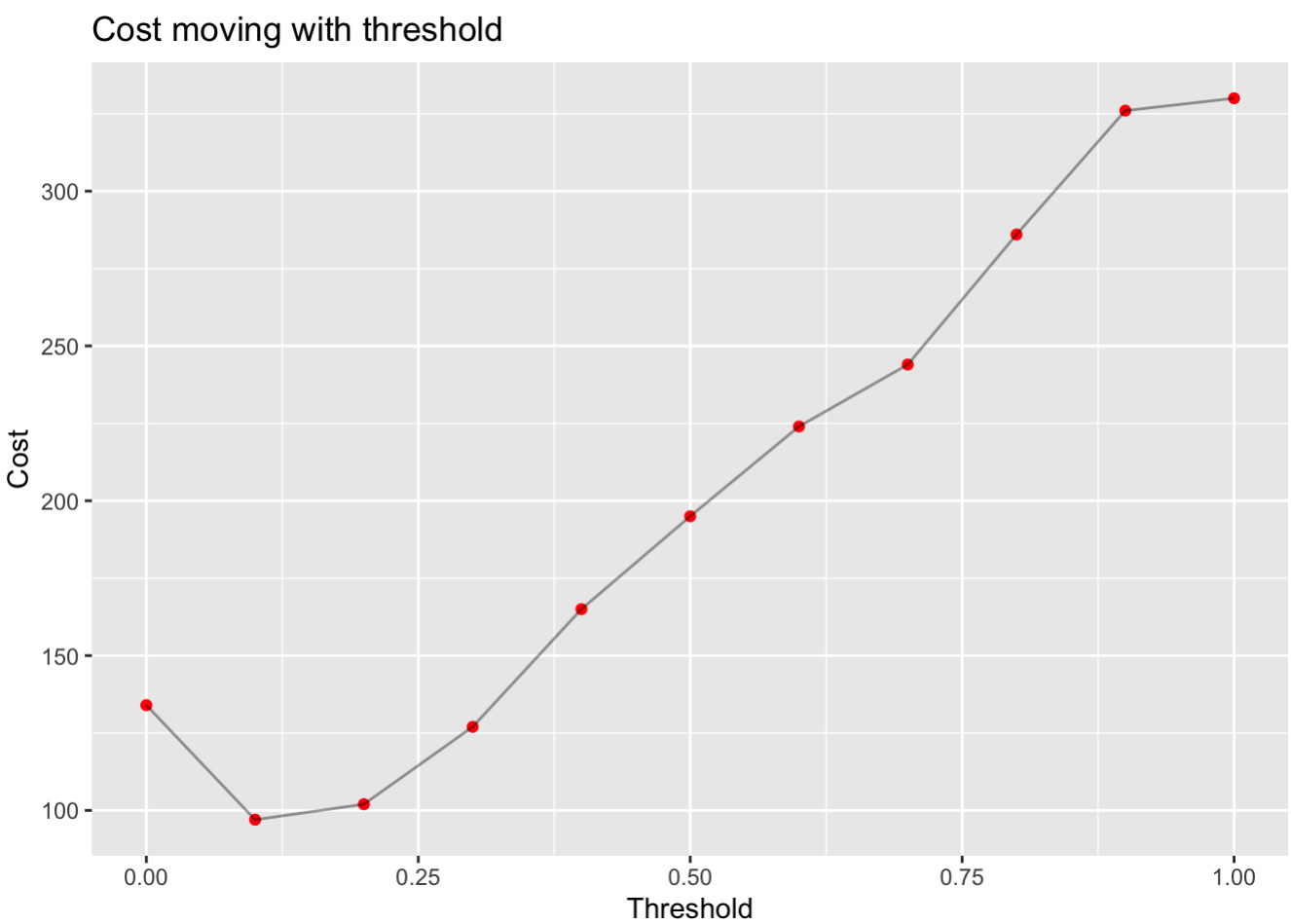
```
step<-seq(0,1,0.1)
step
```

```
##  [1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
```

```
sensitivity_logmodel1<-c()
specificity_logmodel1<-c()
accuracy_logmodel1<-c()
cost_logmodel1<-c()
for(i in step){
  thres=i
  predict1_binary<-as.integer(predict1>thres)
  confuse_matrix<-as.matrix(table(predict1_binary,test$V21))
  if(nrow(confuse_matrix)==2) {
    sensitivity<-confuse_matrix[1,1]/sum(confuse_matrix[,1])
  }else if(thres>0.5){
    sensitivity=1
  }else(sensitivity=0)

  if(nrow(confuse_matrix)==2)
    #confuse matrix will not form as 2*2 with extreme boundary, need to specify!
  {specificity<-confuse_matrix[2,2]/sum(confuse_matrix[,2])
  }else if(thres>0.5){
    specificity=0}else{specificity=1}

  accuracy<-sum(diag(confuse_matrix))/sum(confuse_matrix)
  if(nrow(confuse_matrix)==2){cost<-confuse_matrix[1,2]*5+confuse_matrix[2,1]
    }else if(thres>0.5){cost=confuse_matrix[1,2]*5}else{cost=confuse_matrix[1,1]}
  # This is the part that makes people confused with "confuse matrix". Draw a picture is a good idea
l:)

  sensitivity_logmodel1<-c(sensitivity_logmodel1,sensitivity)
  specificity_logmodel1<-c(specificity_logmodel1,specificity)
  accuracy_logmodel1<-c(accuracy_logmodel1,accuracy)
  cost_logmodel1<-c(cost_logmodel1,cost)
}

df_logmodel1<-data.frame(threshold=step,sensitivity=sensitivity_logmodel1,
                         specificity=specificity_logmodel1,
                         accuracy=accuracy_logmodel1,cost=cost_logmodel1)
df_logmodel1
```

```
##    threshold sensitivity specificity accuracy cost
## 1        0.0   0.0000000  1.00000000    0.670  134
## 2        0.1   0.4626866  0.92424242    0.615   97
## 3        0.2   0.6492537  0.83333333    0.710  102
## 4        0.3   0.7611940  0.71212121    0.745  127
## 5        0.4   0.8134328  0.57575758    0.735  165
## 6        0.5   0.8880597  0.45454545    0.745  195
## 7        0.6   0.9328358  0.34848485    0.740  224
## 8        0.7   0.9701493  0.27272727    0.740  244
## 9        0.8   0.9925373  0.13636364    0.710  286
## 10       0.9   0.9925373  0.01515152    0.670  326
## 11       1.0   1.0000000  0.00000000    0.670  330
```


Sensitivity, specificity and accuracy moving with threshold
Sensitivity
Accuracy
Specificity
Value
Threshold
1.00
0.75
0.50
0.25
0.00
0.00
0.25
0.50
0.75
1.00


Cost moving with threshold
Cost
Threshold
300
250
200
150
100
0.00
0.25
0.50
0.75
1.00

Threshold and sensitivity are positively correlated with each other. Sensitivity is negatively correlated with specificity.

Based on our observation, for log_model1, if we mainly focus on the prediction accuracy, thresholds between 0.3~0.7 are quite similar. Sensitivity and specificity lines cross around each other between the thresholds 0.2 and 0.3 which means for these 2 thresholds, the sensitivity and specificity and balanced, neither one is too high or too low.

However, the cost of FP is 5 times the cost of FN. From above we can see the lower the threshold, the less the loss, this is because if we set the threshold very low, we will categorize most of the customers as 1 which means they are “bad” to protect the bank itself.

When threshold=0.3, the cost is relatively low. Sensitivity and specificity are beautifully balanced.

Logistic Regression Model with Statistical Significant Predictors

From the output from log_model1, we can see there are many coefficients that are not statistical significant. Let’s eliminate some of the predictors and build a new model. But need to keep in mind **If we decide to keep only predictors with p-values lower than a threshold (normally=0.05), we have to keep the whole categorical predictor if any level inside it is significant.**

Build log_model2

```
log_model2<-glm(V21~V1+V2+V3+V4+V5+V6+V8+V9+V10+V13+V14,family=binomial(link = "logit"),data=train)
summary(log_model2)
```

```
##
## Call:
## glm(formula = V21 ~ V1 + V2 + V3 + V4 + V5 + V6 + V8 + V9 + V10 +
##     V13 + V14, family = binomial(link = "logit"), data = train)
##
## Deviance Residuals:
##     Min       1Q   Median       3Q      Max
## -2.1890  -0.7209  -0.3862   0.7154   3.0934
##
## Coefficients:
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept)  2.070e+00  8.022e-01   2.580 0.009877 **
## V1A12       -2.572e-01  2.345e-01  -1.097 0.272751
## V1A13       -7.821e-01  4.016e-01  -1.948 0.051459 .
## V1A14       -1.763e+00  2.541e-01  -6.937    4e-12 ***
## V2           2.713e-02  9.659e-03   2.809 0.004966 **
## V3A31       -6.850e-01  6.017e-01  -1.138 0.254957
## V3A32       -1.185e+00  4.742e-01  -2.499 0.012439 *
## V3A33       -1.268e+00  5.385e-01  -2.355 0.018511 *
## V3A34       -1.776e+00  4.959e-01  -3.581 0.000342 ***
## V4A41       -1.371e+00  3.951e-01  -3.469 0.000522 ***
## V4A410      -1.488e+00  8.000e-01  -1.860 0.062905 .
## V4A42       -7.659e-01  2.823e-01  -2.713 0.006658 **
## V4A43       -1.047e+00  2.709e-01  -3.864 0.000112 ***
## V4A44       -1.360e+00  9.130e-01  -1.489 0.136373
## V4A45       -9.271e-01  6.218e-01  -1.491 0.135998
## V4A46        2.850e-01  4.184e-01   0.681 0.495727
## V4A48       -1.536e+01  4.822e+02  -0.032 0.974582
## V4A49       -1.245e+00  3.774e-01  -3.299 0.000969 ***
## V5           8.464e-05  4.470e-05   1.894 0.058269 .
## V6A62       -2.335e-01  3.222e-01  -0.725 0.468724
## V6A63       -5.310e-01  4.710e-01  -1.127 0.259586
## V6A64       -1.583e+00  6.678e-01  -2.370 0.017793 *
## V6A65       -9.090e-01  2.796e-01  -3.251 0.001150 **
## V8           3.032e-01  9.600e-02   3.158 0.001589 **
## V9A92       -1.417e-01  4.181e-01  -0.339 0.734692
## V9A93       -6.981e-01  4.092e-01  -1.706 0.088015 .
```

```
## V9A94       -4.974e-01  5.074e-01  -0.980 0.326901
## V10A102      8.137e-01  4.412e-01   1.844 0.065133 .
## V10A103     -9.263e-01  4.573e-01  -2.026 0.042808 *
## V13         -2.441e-02  9.243e-03  -2.641 0.008277 **
## V14A142     -4.802e-02  4.611e-01  -0.104 0.917064
## V14A143     -7.004e-01  2.625e-01  -2.668 0.007638 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 967.00  on 799  degrees of freedom
## Residual deviance: 726.15  on 768  degrees of freedom
## AIC: 790.15
##
## Number of Fisher Scoring iterations: 14
```

We can see at this time, the AIC decrease from 809.27 to 790.2 which is better.

Calculate ROC and AUC for log_model2

```
predict2<-predict(log_model2,test,type="response")
roc_log_model2<-roc(test$V21,predict2)
roc_log_model2
```

```
##
## Call:
## roc.default(response = test$V21, predictor = predict2)
##
## Data: predict2 in 134 controls (test$V21 0) < 66 cases (test$V21 1).
## Area under the curve: 0.7872
```

We can see the AUC didn't improve at all for this model. So let's keep moving on to eliminates V5 and V9.

Build log_model3

```
log_model3<-glm(V21~V1+V2+V3+V4+V6+V8+V10+V13+V14,family=binomial(link = "logit"),data=train)
summary(log_model3)
```

```
##
## Call:
## glm(formula = V21 ~ V1 + V2 + V3 + V4 + V6 + V8 + V10 + V13 +
##     V14, family = binomial(link = "logit"), data = train)
##
## Deviance Residuals:
##     Min       1Q   Median       3Q      Max
## -2.0441  -0.7127  -0.3982   0.7416   3.0086
##
## Coefficients:
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept)   2.036345   0.677070   3.008 0.002633 **
## V1A12        -0.233181   0.231658  -1.007 0.314138
## V1A13        -0.838221   0.397299  -2.110 0.034876 *
## V1A14        -1.760016   0.251182  -7.007 2.44e-12 ***
## V2            0.036683   0.007709   4.758 1.95e-06 ***
## V3A31        -0.769052   0.595775  -1.291 0.196758
## V3A32        -1.234012   0.468140  -2.636 0.008389 **
## V3A33        -1.358896   0.533154  -2.549 0.010810 *
## V3A34        -1.842322   0.491168  -3.751 0.000176 ***
## V4A41        -1.286348   0.386551  -3.328 0.000875 ***
```

```
## V4A410       -1.210947   0.732789  -1.653 0.098429 .
## V4A42        -0.666453   0.276550  -2.410 0.015958 *
## V4A43        -1.039029   0.266427  -3.900 9.62e-05 ***
## V4A44        -1.326863   0.896390  -1.480 0.138812
## V4A45        -0.955528   0.627842  -1.522 0.128028
## V4A46         0.321797   0.415840   0.774 0.439021
## V4A48       -15.451992 469.034409  -0.033 0.973719
## V4A49        -1.244030   0.373746  -3.329 0.000873 ***
## V6A62        -0.249828   0.320532  -0.779 0.435735
## V6A63        -0.569971   0.470995  -1.210 0.226225
## V6A64        -1.556393   0.665774  -2.338 0.019402 *
## V6A65        -0.896579   0.274904  -3.261 0.001109 **
## V8            0.197034   0.085715   2.299 0.021522 *
## V10A102       0.834108   0.434044   1.922 0.054642 .
## V10A103      -1.025475   0.449248  -2.283 0.022451 *
## V13          -0.025987   0.009007  -2.885 0.003910 **
## V14A142      -0.033344   0.459051  -0.073 0.942096
## V14A143      -0.628007   0.259665  -2.419 0.015584 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 967  on 799  degrees of freedom
## Residual deviance: 737  on 772  degrees of freedom
## AIC: 793
##
## Number of Fisher Scoring iterations: 14
```

Calculate ROC and AUC for log_model3

```
predict3<-predict(log_model3,test,type="response")
roc_log_model3<-roc(test$V21,predict3)
roc_log_model3
```

```
##
## Call:
## roc.default(response = test$V21, predictor = predict3)
##
## Data: predict3 in 134 controls (test$V21 0) < 66 cases (test$V21 1).
## Area under the curve: 0.7681
```

We can see this time, both AUC decrease and AIC increase which means this model is worse than before! It seems by eliminating the whole predictors based on the p-value doesn't really improve the performance.

Decompose Levels from Categorical Columns and Build Logistic Regression Model

Use p-value<0.1 as the threshold for coefficients.

```
select_v1<-c('A13','A14') #select all significant levels
select_v3<-c('A32','A33','A34')
select_v4<-c('A41','A410','A42','A43','A49')
select_v6<-c('A64','A65')
select_v9<-c('A93')
select_v10<-c('A103')
select_v14<-c('A143')
lst<-list(select_v1,select_v3,select_v4,select_v6,select_v9,select_v10,
          select_v14)
```

```
names(lst)<-c(1,3,4,6,9,10,14) #rename our list

train1<-train
for (j in c(1,3,4,6,9,10,14)){
  for (i in lst[[as.character(j)]]) {
    vector<-rep(0,800)
    mask<-train[,j]==i
    vector[mask]=1
    train1<-cbind(train1,vector)

  }
}
names(train1)[22:36]<-c('A13','A14','A32','A33','A34','A41','A410','A42','A43','A49','A64','A65','A9
3','A103','A143')
head(train1)
```

```
##    V1 V2  V3  V4   V5  V6  V7 V8  V9  V10 V11  V12 V13  V14  V15 V16  V17 V18
## 1 A11 12 A32 A42  708 A61 A73  2 A93 A103   3 A122  38 A143 A152   1 A172   2
## 2 A14 12 A34 A42 5801 A65 A75  2 A93 A101   4 A122  49 A143 A151   1 A173   1
## 3 A12 18 A32 A49 4439 A61 A75  1 A93 A102   1 A121  33 A141 A152   1 A174   1
## 4 A14 24 A32 A41 7814 A61 A74  3 A93 A101   3 A123  38 A143 A152   1 A174   1
## 5 A14 18 A32 A43 1473 A61 A72  3 A94 A101   4 A121  39 A143 A152   1 A173   1
## 6 A11 48 A32 A40 3931 A61 A74  4 A93 A101   4 A124  46 A143 A153   1 A173   2
##    V19  V20 V21 A13 A14 A32 A33 A34 A41 A410 A42 A43 A49 A64 A65 A93 A103 A143
## 1 A191 A201   0   0   0   1   0   0   0    0   1   0   0   0   0   1    1    1
## 2 A192 A201   0   0   1   0   0   1   0    0   1   0   0   0   1   1    0    1
## 3 A192 A201   0   0   0   1   0   0   0    0   0   0   1   0   0   1    0    0
## 4 A192 A201   0   0   1   1   0   0   1    0   0   0   0   0   0   1    0    1
## 5 A192 A201   0   0   1   1   0   0   0    0   0   1   0   0   0   0    0    1
## 6 A191 A201   1   0   0   1   0   0   0    0   0   0   0   0   0   1    0    1
```

Modify our test dataset:

```
test1<-test
for (j in c(1,3,4,6,9,10,14)){
  for (i in lst[[as.character(j)]]) {
    vector<-rep(0,200)
    mask<-test1[,j]==i
    vector[mask]=1
    test1<-cbind(test1,vector)

  }
}
names(test1)[22:36]<-c('A13','A14','A32','A33','A34','A41','A410','A42','A43','A49','A64','A65','A93'
,'A103','A143')
head(test1)
```

```
##    V1 V2  V3  V4    V5  V6  V7 V8  V9  V10 V11  V12 V13  V14  V15 V16  V17 V18
## 1 A14 24 A32 A42  2835 A63 A75  3 A93 A101   4 A122  53 A143 A152   1 A173   1
## 2 A11 48 A32 A49  4308 A61 A72  3 A92 A101   4 A122  24 A143 A151   1 A173   1
## 3 A12 12 A32 A43  1567 A61 A73  1 A92 A101   1 A123  22 A143 A152   1 A173   1
## 4 A11 24 A32 A43  1282 A62 A73  4 A92 A101   2 A123  32 A143 A152   1 A172   1
## 5 A12 24 A32 A41 12579 A61 A75  4 A92 A101   2 A124  44 A143 A153   1 A174   1
## 6 A12 12 A34 A41  1804 A62 A72  3 A93 A101   4 A122  44 A143 A152   1 A173   1
##    V19  V20 V21 A13 A14 A32 A33 A34 A41 A410 A42 A43 A49 A64 A65 A93 A103 A143
## 1 A191 A201   0   0   1   1   0   0   0    0   1   0   0   0   0   1    0    1
## 2 A191 A201   1   0   0   1   0   0   0    0   0   0   1   0   0   0    0    1
## 3 A192 A201   0   0   0   1   0   0   0    0   0   1   0   0   0   0    0    1
## 4 A191 A201   1   0   0   1   0   0   0    0   0   1   0   0   0   0    0    1
```

```
## 5 A192 A201   1   0   0   1   0   0   1    0   0   0   0   0   0   0    0    1
## 6 A191 A201   0   0   0   0   0   1   1    0   0   0   0   0   0   1    0    1
```

We can see both our training set and test set are with 36 predictors!

Build log_model4

Build our Logistic regression model with this new training dataset (train1) and test by the new test dataset (test1)

```
log_model4<-glm(V21~A13+A14+A32+A33+A34+A41+A410+A42+A43+A49+A64+A65+A93+A103+
                  A143+V2+V5+V8+V13,family=binomial(link ='logit'),data=train1)
summary(log_model4)
```

```
##
## Call:
## glm(formula = V21 ~ A13 + A14 + A32 + A33 + A34 + A41 + A410 +
##     A42 + A43 + A49 + A64 + A65 + A93 + A103 + A143 + V2 + V5 +
##     V8 + V13, family = binomial(link = "logit"), data = train1)
##
## Deviance Residuals:
##     Min       1Q   Median       3Q      Max
## -2.0046  -0.7449  -0.4051   0.7946   3.0328
##
## Coefficients:
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept)  8.689e-01  5.405e-01   1.608 0.107916
## A13         -5.696e-01  3.762e-01  -1.514 0.129991
## A14         -1.680e+00  2.199e-01  -7.643 2.13e-14 ***
## A32         -6.606e-01  3.124e-01  -2.115 0.034452 *
## A33         -7.993e-01  4.073e-01  -1.962 0.049709 *
## A34         -1.177e+00  3.435e-01  -3.426 0.000612 ***
## A41         -1.319e+00  3.796e-01  -3.474 0.000513 ***
## A410        -1.149e+00  7.706e-01  -1.491 0.136035
## A42         -5.444e-01  2.595e-01  -2.098 0.035893 *
## A43         -9.588e-01  2.458e-01  -3.900 9.61e-05 ***
## A49         -1.089e+00  3.475e-01  -3.134 0.001724 **
## A64         -1.496e+00  6.526e-01  -2.293 0.021836 *
## A65         -8.815e-01  2.662e-01  -3.312 0.000927 ***
## A93         -4.645e-01  1.940e-01  -2.394 0.016660 *
## A103        -9.632e-01  4.440e-01  -2.170 0.030043 *
## A143        -6.762e-01  2.297e-01  -2.943 0.003246 **
## V2           2.692e-02  9.490e-03   2.837 0.004559 **
## V5           1.052e-04  4.332e-05   2.428 0.015168 *
## V8           2.972e-01  9.292e-02   3.199 0.001379 **
## V13         -2.063e-02  8.906e-03  -2.317 0.020527 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 967.00  on 799  degrees of freedom
## Residual deviance: 749.98  on 780  degrees of freedom
## AIC: 789.98
##
## Number of Fisher Scoring iterations: 5
```

We can see most of our coefficients are significant! The AIC=790 which is not a big decrease compare with other models we build by original predictors.

Predict test1 Data by log_model4:

```
predict4<-predict(log_model4,test1,type="response")
head(predict4)
```

```
##          1          2          3          4          5          6
## 0.08206743 0.64221391 0.25060248 0.47080222 0.61400577 0.09358258
```

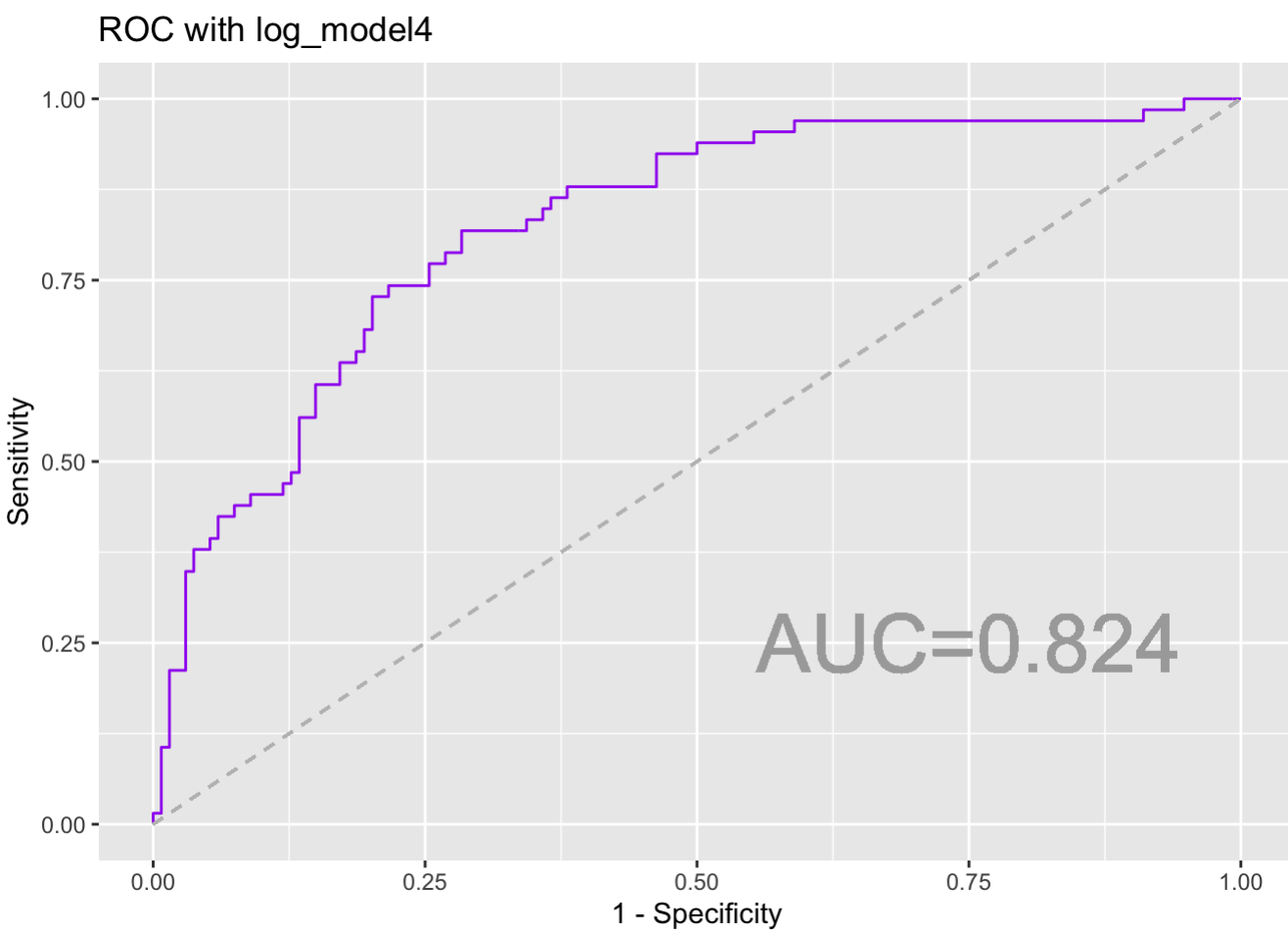
Calculate ROC and AUC for log_model4

```
roc_log_model4<-roc(test1$V21,predict4)
roc_log_model4
```

```
##
## Call:
## roc.default(response = test1$V21, predictor = predict4)
##
## Data: predict4 in 134 controls (test1$V21 0) < 66 cases (test1$V21 1).
## Area under the curve: 0.8241
```

The AUC increased a lot from mainly around 0.78 to 0.824. I like the improvement a lot. Let's take a look at the ROC curve.

Draw ROC Curve:



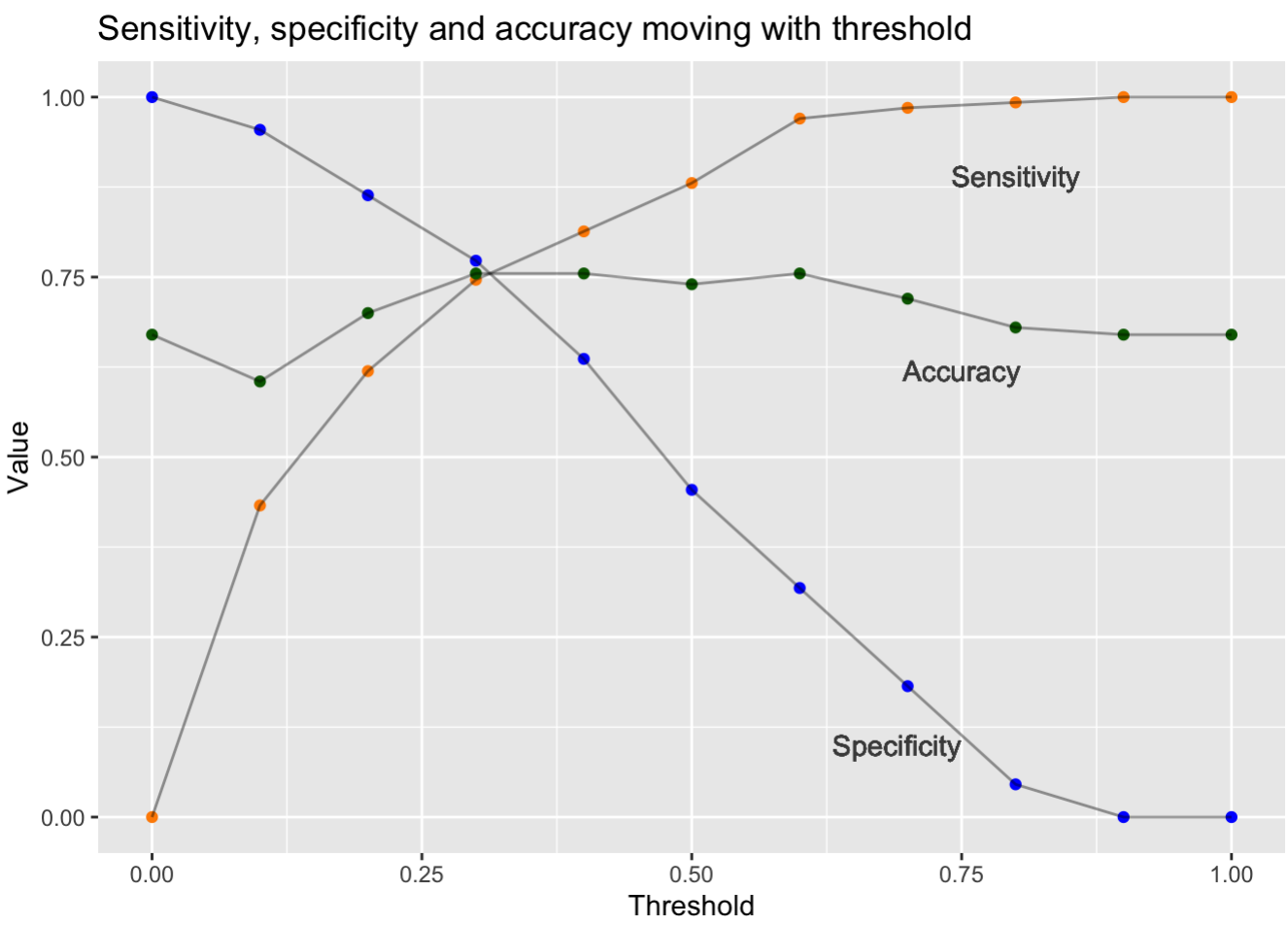
Build a Loop for the Performance of log_model4:

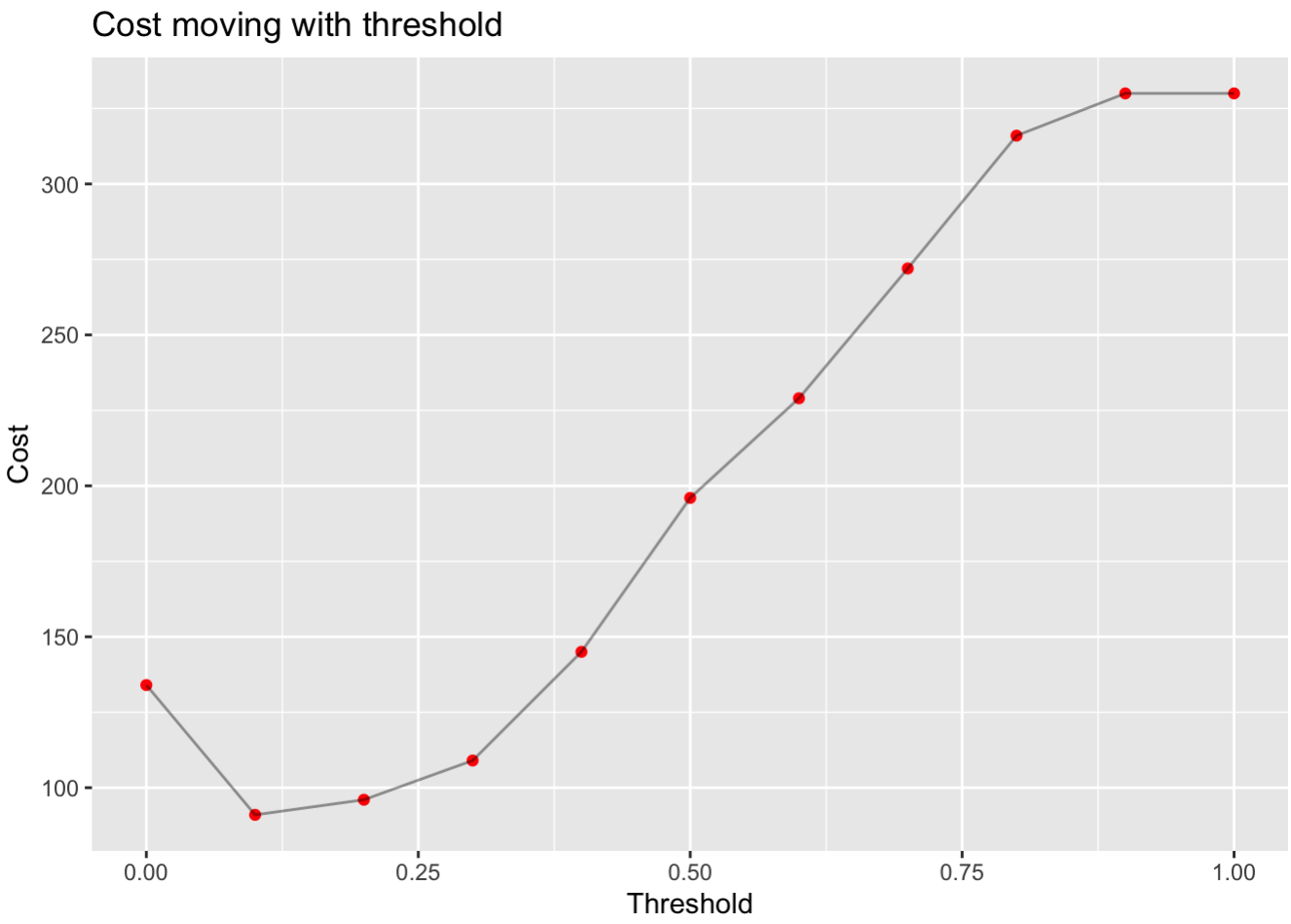
```
step<-seq(0,1,0.1)
sensitivity_logmodel4<-c()
```

```r
specificity_logmodel4<-c()
accuracy_logmodel4<-c()
cost_logmodel4<-c()

for(i in step){
  thres=i
  predict4_binary<-as.integer(predict4>thres)
  confuse_matrix<-as.matrix(table(predict4_binary,test1$V21))
  if(nrow(confuse_matrix)==2) {
    sensitivity<-confuse_matrix[1,1]/sum(confuse_matrix[,1])
  }else if(thres>0.5){
    sensitivity=1
  }else(sensitivity=0)

  if(nrow(confuse_matrix)==2)
    #confuse matrix will not form as 2*2 with extreme boundary, need to specify!
  {specificity<-confuse_matrix[2,2]/sum(confuse_matrix[,2])
  }else if(thres>0.5){
    specificity=0}else{specificity=1}

  accuracy<-sum(diag(confuse_matrix))/sum(confuse_matrix)
  if(nrow(confuse_matrix)==2){cost<-confuse_matrix[1,2]*5+confuse_matrix[2,1]
    }else if(thres>0.5){cost=confuse_matrix[1,2]*5}else{cost=confuse_matrix[1,1]}

  sensitivity_logmodel4<-c(sensitivity_logmodel4,sensitivity)
  specificity_logmodel4<-c(specificity_logmodel4,specificity)
  accuracy_logmodel4<-c(accuracy_logmodel4,accuracy)
  cost_logmodel4<-c(cost_logmodel4,cost)
}

df_logmodel4<-data.frame(threshold=step,sensitivity=sensitivity_logmodel4,
                         specificity=specificity_logmodel4,
                         accuracy=accuracy_logmodel4,cost=cost_logmodel4)
df_logmodel4
```

```
##    threshold sensitivity specificity accuracy cost
## 1        0.0   0.0000000  1.00000000    0.670  134
## 2        0.1   0.4328358  0.95454545    0.605   91
## 3        0.2   0.6194030  0.86363636    0.700   96
## 4        0.3   0.7462687  0.77272727    0.755  109
## 5        0.4   0.8134328  0.63636364    0.755  145
## 6        0.5   0.8805970  0.45454545    0.740  196
## 7        0.6   0.9701493  0.31818182    0.755  229
## 8        0.7   0.9850746  0.18181818    0.720  272
## 9        0.8   0.9925373  0.04545455    0.680  316
## 10       0.9   1.0000000  0.00000000    0.670  330
## 11       1.0   1.0000000  0.00000000    0.670  330
```

Let's visualize the specificity, the sensitivity, and the accuracy.

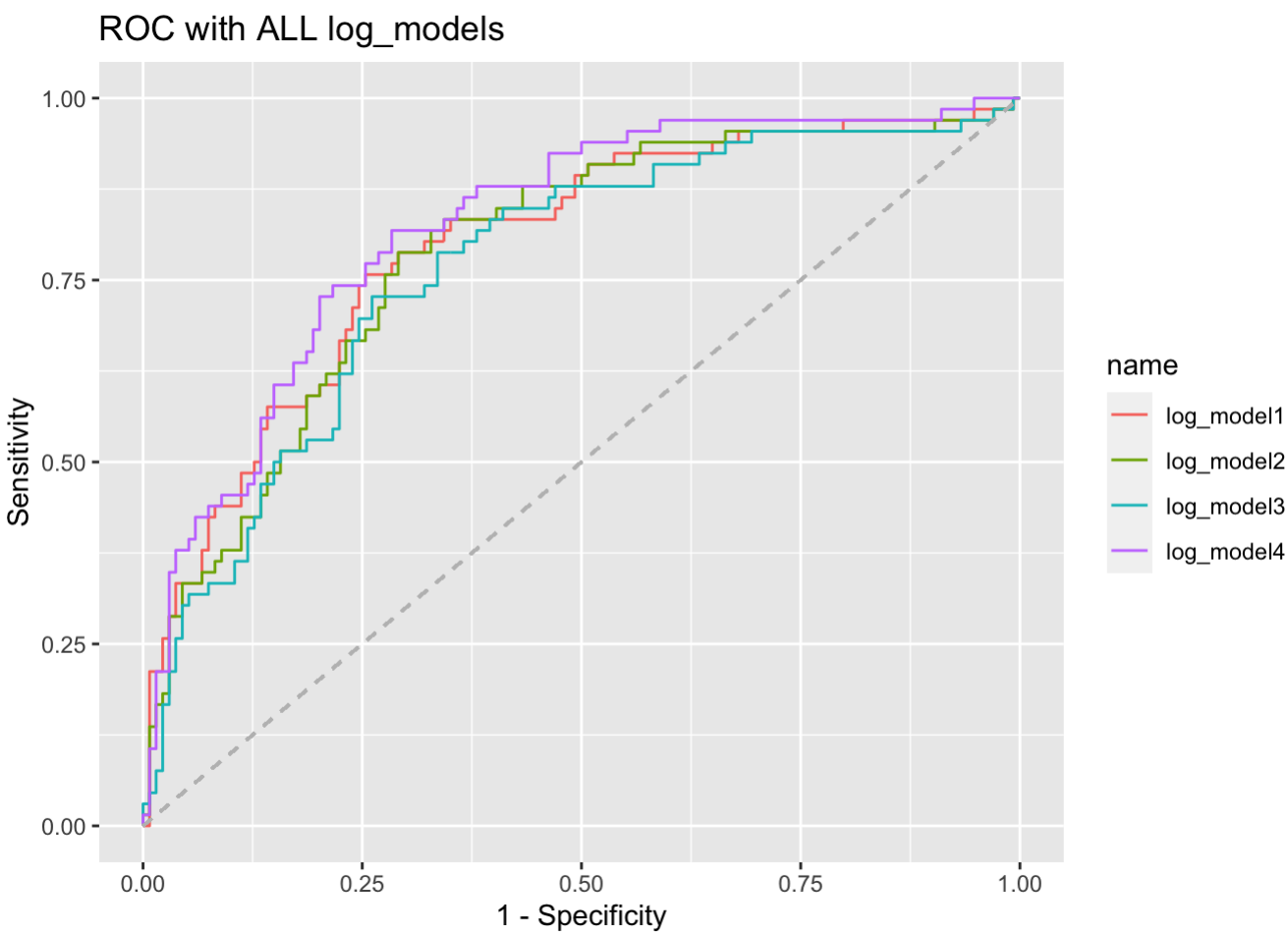

Sensitivity, specificity and accuracy moving with threshold
Sensitivity
Accuracy
Specificity
Value
Threshold
1.00
0.75
0.50
0.25
0.00
0.00
0.25
0.50
0.75
1.00



From above we can tell the overall cost has decreased a little compared with log_model1 which is good.

The accuracy of log_model4 is relatively higher at threshold=0.3, also at this point, specificity and sensitivity come across with each other. If we focus mainly on decreasing the cost, we should choose threshold = 0.1, however, if we want to get a threshold that balances each index, I will suggest choosing threshold=0.3.

Let's plot all of the ROC curves to visualize which model is the best with the largest AUC:



We can see the purple ROC curve represents our log_model4 is with the biggest AUC. With setting threshold=0.1, the cost turns to be the lowest=91 among all logistic models, but the accuracy rate is lower than average.

Compare all models' AIC & AUC:

Index	log_model1	log_model2	log_model3	log_model4
AIC	809.3000	790.1500	793.0000	789.9800
AUC	0.7974	0.7872	0.7681	0.8241

Conclusion

- There are many categorical columns in the German Credit dataset. The glm() function can automatically apply one-hot encoding to interpret each level as a new predictor for the final response.
- If we want to use individual levels that are significant as predictors in our logistic regression model, we need to manually extract the level one by one from our original dataset.
- The best model turns out to be log_model4 with predictors: A13+A14+A32+A33+A34+A41+A410+A42+A43+A49+A64+A65+A93+A103+A143+V2+V5+V8+V13.

It's AUC=0.824 (the biggest among all models) and AIC=789.980 (smallest). The threshold I choose for log_model4 is 0.1 with the lowest cost=91 since we mainly focus on decreasing the cost other than balance the sensitivity and specificity. The accuracy from our test dataset = 0.605 which is not very high. But for decreasing the loss, we need to compromise with more FN.